Umeå University, Sweden

Department of Applied Physics and Electronics

MSc in Interaction Technology and Design, 300 ECTS

Master Thesis, 30 ECTS

Estimating the Estimate: Using Machine Learning to Forecast Train Delay Reliability in Real Time

Jakob Gottfridsson

External Supervisor:	Simon Palm
Academic Supervisor:	Thomas Mejtoft
Examiner:	Ulrik Söderström
Collaborator:	Framna AB

June 30, 2026

Abstract

Real time ETAs (Estimated Time of Arrival/departure) published by railway operators such as Trafikverket revise frequently between issuance and realisation, but the operational system does not quantify how much each revision is likely to still be wrong. This gap matters for both passengers and operators, wanting trust for decision-making and being able to reroute and manage connections, respectively.

Most prior data-driven delay forecasting work either competes with the operational baseline from scratch or studies one model and one interval construction method in isolation. This thesis instead frames delay forecasting as a correction and improvement of the operational ETA and compares three model families on the same task, evaluating both point accuracy and the calibration of their native prediction intervals.

A snapshot-keyed dataset was constructed by collecting Trafikverket's API every five minutes for one month (2026-02-02 to 2026-03-02), recording all state changes to SJ long-distance ETAs along with the realised outcome. Ridge regression, Random Forest and LightGBM were trained to predict realised delay at each snapshot, with the operational ETA included as a feature, both as real time delay estimation and historical ETAs, so that improvements can be interpreted as corrections to the operational forecast. Models were then compared on common accuracy metrics (MAE, $RMSE$ and R^2) as well as on skill against the operational ETA, with results stratified by forecast horizon and delay severity, and 90% prediction intervals evaluated by empirical coverage.

All three models improved on the operational ETA on $RMSE$, with skill ranking by complexity (+0.10 for Ridge, +0.21 for Random Forest, +0.24 for LightGBM). Skill was concentrated in long horizons and severe delays. In the on-time bin, models added no value over the baseline. ETA history features dominated importance rankings, and weather features were essentially uninformative. The three interval construction methods produced qualitatively distinct failure modes. LightGBM's quantile regression achieved near nominal coverage (90.7%), Random Forest's tree-prediction-percentile severely under-covered (27.6%), and Ridge's independent quantile fits produced frequent

bound crossings.

The findings establish that real time API snapshots, using the operational systems in place, can both improve point predictions and communicate calibrated uncertainty, given the right model and interval construction. The snapshot-keyed schema and the skill versus baseline framing are transferable to other domains with revising ETAs. Results are conditional on a single winter month and a single operator, motivating future work in multi-season collection and conformal calibration.

Sammanfattning

Beräknade ankomsttider (ETA:er) i realtid, publicerade av järnvägsoperatörer som Trafikverket, uppdateras frekvent mellan utfärdande och faktiskt ankomst, men systemet kvantifierar inte hur stor risk det är för revideringen att fortfarande vara fel. Denna brist är viktig för både passagerare, som vill kunna lita på systemet och fatta beslut, och operatörer, som vill kunna hantera spårändringar och tågbyten.

De flesta tidigare arbeten om datadrivna förseningsprognoser tävlar antingen med den operativa baslinjen från början eller studerar en isolerad modell och en isolerad intervallskonstruktion. Det här arbetet ramar istället in förseningsprognoser som en korrektion och förbättring av den operativa ETA:en, och jämför tre modell-familjer på samma uppgift, att utreda både punktnoggrannhet och kalibreringen av deras inbyggda prediktionsintervaller.

Ett snapshot-kodat dataset konstruerades genom att hämta Trafikverkets API var femte minut under en månad (2026-02-02 till 2026-03-02), och fånga upp alla tillståndsändringar av ETA:er från SJ:s långdistanståg tillsammans med deras realiserade utfall. Ridge regression, Random Forest och LightGBM tränades för att kunna prognostisera realiserad försening för varje snapshot, med Trafikverkets ETA inkluderad som en feature både som senaste ETA-försening och som historiska ETA-värden, så att förbättringar kan tydas som korrektioner av den operativa prognosen. Modellerna jämfördes sedan genom ordinära felmetriker (MAE, $RMSE$ och R^2) och genom *skill* gentemot den operativa ETA:en, med resultat stratifierade genom prognoshorisont och förseningsgrad, samt 90-procents-prediktionsintervaller utvärderade med empirisk täckning.

Alla tre modeller förbättrade den operativa ETA:en på $RMSE$, med skill som följer modellkomplexitet (+0,10 för Ridge, +0,21 för Random Forest och +0,24 för LightGBM). Färdigheten var koncentrerad över långa horisonter och allvarliga förseningar. För tåg i tid gav inte modellerna något ytterligare värde över baslinjen. Historiska ETA-features dominerade viktrankingar, och väder-features var i princip oinformativa. De tre intervallkonstruktionsmetoderna producerade tre kvalitativt distinkta

fellägen. LightGBM:s kvantilregression åstadkom en nära nominell täckning (90,7%), medan Random Forest:s trädprediktions-percentil undertäckte rejält (27,6%), och Ridge oberoende kvantilpassningar producerade frekventa gränsöverskridningar.

Dessa fynd visar att realtids-snapshots som använder de operativa systemen som redan finns både kan förbättra punktprediktioner och kommunicera kalibrerad osäkerhet, givet rätt modell och intervallskonstruktion. Det snapshot-kodade schemat och inramningen av "skill mot baslinje" är båda överförbara till andra domän med uppdaterande ETA:er. Resultaten är betingade av en enstaka vintermånad och en enstaka tågoperatör, vilket motiverar framtida arbeten med flersäsongshämtning och konformal kalibrering.

Acknowledgements

I would like to thank the amazing people at Framna for letting me spend a semester working on this thesis at their offices. It has been great getting to know you all and I feel very excited getting to come back after the summer vacation. A special thanks goes to Simon Palm, for pushing me to get things done and being a great person to discuss ideas and resources with. Another special thanks goes to Josephine Holmström, for checking in with me and making sure I had everything I needed to be able to finish the thesis.

I would also like to thank Thomas Mejtoft for guiding me through the writing process, giving feedback along the way, and making sure I stayed on schedule. His presence in my 5 years at this programme has granted me the knowledge and self-belief to feel secure in my skills as a soon to be graduated interaction designer.

On a personal note, I would like to thank Rebecka for being my number one supporter during this stressful time for the both of us. Thank you for believing in me when I'm doubting myself, reading my drafts when I'm particularly proud of something I wrote, and loving me for who I am.

To the friends I've made during these 5 years, thank you for showing me what community and friendship should look like (and for buying tickets to my shows).

Lastly, I want to thank my family for supporting me no matter what. You mean the world to me.

Contents

Abstract ii

Sammanfattning iv

Acknowledgements v

List of Abbreviations xiii

1 Introduction 1
- 1.1 Objectives . . . 2
  - 1.1.1 Proof-of-concept framing . . . 3
- 1.2 Contributions . . . 3
- 1.3 External Partner . . . 3

2 Background & Theoretical Framework 4
- 2.1 Railway Delays and Delay Propagation . . . 4
  - 2.1.1 Delay definition . . . 4
  - 2.1.2 Network Effects . . . 5
  - 2.1.3 Propagation Models . . . 6
  - 2.1.4 Forecasting Challenges . . . 7
- 2.2 ETA Systems and Trafikverket Open API . . . 8
- 2.3 Machine Learning for Forecasting . . . 8
  - 2.3.1 Supervised Learning for Regression . . . 9
  - 2.3.2 Candidate Model Families . . . 9
  - 2.3.3 Loss Functions . . . 17
  - 2.3.4 Hyperparameter Optimisation . . . 18
  - 2.3.5 Challenges . . . 22
  - 2.3.6 Feature Importance and Prediction Uncertainty as Interpretability 23
- 2.4 Related Work . . . 26
  - 2.4.1 Analytical Propagation Models . . . 27

2.4.2 Data-Driven Delay Prediction . . . 27
2.4.3 Uncertainty-Aware Delay Forecasting . . . 28
2.4.4 Thesis Positioning . . . 29

3 Methodology **30**
3.1 Data Collection . . . 30
3.2 Dataset Construction and Labelling . . . 32
3.3 Features . . . 34
3.4 Models . . . 35
3.4.1 Ridge . . . 35
3.4.2 LightGBM . . . 36
3.4.3 Random Forest . . . 36
3.5 Evaluation Protocol . . . 36
3.5.1 Training and Validation Partition . . . 37
3.5.2 Hyperparameter Optimisation . . . 37
3.5.3 Error Metrics . . . 38
3.5.4 Feature Importance . . . 38
3.5.5 Result Stratifications . . . 38
3.5.6 Prediction Interval . . . 39
3.6 Ethics and Data Considerations . . . 39
3.7 Implementation notes . . . 40
3.7.1 Programming Languages . . . 40
3.7.2 Hardware & Runtime Environment . . . 40

4 Results **41**
4.1 Descriptive Statistics . . . 41
4.1.1 Dataset composition . . . 41
4.1.2 Snapshot density . . . 41
4.1.3 Actual delay distribution . . . 42
4.1.4 Forecast horizon distribution . . . 43
4.1.5 Trafikverket baseline behaviour . . . 44
4.1.6 Summary . . . 45
4.2 Model Performance . . . 46
4.2.1 Aggregate Point-Prediction Performance . . . 46
4.2.2 Prediction Interval Performance . . . 47
4.2.3 Summary . . . 48
4.3 Error Analysis . . . 48
4.3.1 Residual Distribution . . . 48
4.3.2 Error by Forecast Horizon . . . 49

4.3.3 Error by Delay Magnitude . . . 51
4.3.4 Summary . . . 52
4.4 Feature Importance . . . 53
4.4.1 Permutation Importance . . . 53
4.4.2 SHAP Values . . . 54
4.4.3 Summary . . . 58

5 Discussion **59**
5.1 Interpretation of Results . . . 59
5.1.1 Predictive Accuracy . . . 59
5.1.2 Features and Modelling . . . 62
5.1.3 Prediction Interval Implications . . . 64
5.2 Practical Implications . . . 66
5.2.1 Passenger-Facing Communication . . . 66
5.2.2 Operational Value . . . 66
5.2.3 Field Insights . . . 67
5.3 Limitations . . . 67
5.3.1 Proof-of-Concept Scope . . . 67
5.3.2 Leakage Discovery . . . 68
5.3.3 Long-Horizon Skill as an Upper Bound . . . 68
5.3.4 Class Imbalance of Severe Delays . . . 68
5.3.5 Snapshot Density . . . 68
5.3.6 Categorical Encoding Asymmetry . . . 69
5.3.7 Ensemble Hyperparameter Space . . . 69
5.4 Future Work . . . 69
5.4.1 Extending the Dataset . . . 69
5.4.2 Modelling Extensions . . . 70
5.4.3 Uncertainty and Prediction Intervals . . . 70
5.4.4 Deployment and Communication . . . 71
5.4.5 Generalisation Beyond Rail . . . 71

6 Conclusion **72**
6.1 Answers to the Research Questions . . . 72
6.2 Reflection on Contributions . . . 73
6.3 Closing Remarks . . . 73

A Additional Tables and Figures **81**

B Example Gradient Boosting Process **89**

C Loss Functions Used in Classification **91**

List of Figures

2.1 Example of a decision tree structure. 12
2.2 Example of decision nodes in a decision tree. 12
2.3 Grid search example with two hyperparameters. 19
2.4 Random Search example with two hyperparameters. 20
2.5 Bayesian Optimisation example with two hyperparameters. 21

3.1 Data collection schema. 32

4.1 Distribution of observed states per activity in the validation set. 42
4.2 Distribution of realised delays in the validation set. 43
4.3 Distribution of forecast horizon across the validation set. (a) – Fine-grained histogram with 5-minute bins. (b) – Histogram with analytical bins used throughout this chapter. 44
4.4 Distribution of predicted delay from Trafikverket across the validation set. 45
4.5 Distribution of model residuals. 49

C.1 Example of an AUC-ROC function plot. 93

List of Tables

3.1 Variables captured by the collection scripts. 31
3.2 Feature groupings. 34
3.3 Random Search: chosen hyperparameter values (LightGBM) 37
3.4 Random Search: chosen hyperparameter values (Random Forest) 38
3.5 Grid Search: chosen hyperparameter values (Ridge) 38

4.1 Summary of descriptive statistics on validation set. 46
4.2 Aggregate point-prediction error metrics. 46
4.3 Prediction Interval Diagnostics (90% nominal, [5%, 95%] quantile). . . 47
4.4 Error by forecast horizon - LightGBM. 50
4.5 Error by forecast horizon - Random Forest. 50
4.6 Error by forecast horizon - Ridge. 50
4.7 Error by delay magnitude - LightGBM. 51
4.8 Error by delay magnitude - Random Forest. 51
4.9 Error by delay magnitude - Ridge. 51
4.10 Permutation Importance (5000 samples) - LightGBM. 53
4.11 Permutation Importance (5000 samples) - Random Forest. 53
4.12 Permutation Importance (5000 samples) - Ridge. 54
4.13 Local SHAP explanations – LightGBM (base value = 9.65 min). 55
4.14 Local SHAP explanations – Random Forest (base value = 4.15 min). . 56
4.15 Local SHAP explanations – Ridge (base value = 1.39 min). 57

A.1 Meteorological parameters used from SMHI. 81
A.2 Python libraries used, and their area of usage. 82
A.3 Permutation Importance (5000 samples) – LightGBM (full list). 83
A.4 Permutation Importance (5000 samples) – Random Forest (full list). . . 83
A.5 Permutation Importance (5000 samples) – Ridge (full list). 86

B.1 Example training data 89
B.2 Example error calculation 89

B.3 Example formula application 90

List of Abbreviations

ML — Machine Learning

ETA — Estimated Time of Arrival or departure

ATA — Advertised Time of Arrival or departure

RTA — Realised Time of Arrival or departure

API — Application Programming Interface

MAE — Mean Absolute Error

RMSE — Root Mean Squared Error

CV — Cross-Validation

SHAP — SHapley Additive exPlanations

SMHI — Sveriges Meteorologiska och Hydrologiska Institut (Sweden's Meteorological and Hydrological Institute)

SQL — Structured Query Language

Chapter 1

Introduction

Train delays in Sweden are common and persistent [41] – about 1 in 10 of trains are at least five minutes late. Research shows [8, 39] that delays are a source of irritation and stress for passengers and rail personnel alike, and the delay of one train usually leads to escalation in delay risks for following trains and stations [11], making it even more difficult to maintain a reliable arrival and departure schedule. Reliable travel information in railway systems is of great importance when it comes to gaining passenger trust, minimising rail work and optimising costs, which in turn makes not just delay prevention but delay communication essential [27].

One challenge today is that it is rare to find a single identifiable cause to railway delays [12]. More often than not, many factors emerge and contribute to delays. Accidents, weather, train types and domino effects of previous incidents could all factor in to one delay [27]. It is therefore difficult to isolate causes of delays, especially when trying to single only one out. Delays are also dynamic when it comes to escalation [11]. Events from one place and time could have effect on delays multiple hours in the future and multiple kilometres away. This complexity makes forecasting challenging, even with proper data collection and pattern recognition.

The Swedish Transport Administration (Trafikverket) provides an open API containing railway information data, including real time estimated time-of-arrival (ETA) data [40]. Here, information can be fetched about certain trains at certain stations, such as an estimation of when that train will arrive at or departure from that station, as well as realised times for when it does. It is possible to find the error margin of the ETA by comparing it to the realised arrival/departure time, in order to calculate how many minutes off the ETA was at a certain point in time. The ETA gives an estimation, but limitations emerge when asking questions in real time such as "*how reliable is the current estimation?*" and "*is the ETA subject to change?*". No measure of confidence

or accuracy is given in the API, which makes for an interesting gap in communication and reliability to explore.

While current systems provide updated ETAs, they do not quantify the expected deviation or uncertainty of those predictions [40]. There is a lack of transparency when it comes to escalation likelihood, potentially harming passenger experience and trust. In addition, operators can benefit from an improved transparency of escalation likelihood and uncertainty, granting the ability to plan better and more accurate for route and station congestion alike [27].

In this potential gap for improved reliability and communication, there is an opportunity for Machine Learning (ML). ML has a unique ability to detect and recognise hidden patterns in raw data. Learning relationships between intermediate estimates and realised delays in the data from Trafikverket's API, ML can capture systematic bias and escalation patterns to find what affects ETA score and essentially strengthen this communicative data point with reliability estimations. Compared to rule-based approaches, ML can capture patterns not detectable by logic alone, and can provide more complex recognition solutions, making it suitable for this task.

1.1 Objectives

This thesis aims to explore how ML can be used to improve communication and estimation in operational railway delay forecasting by predicting delays with uncertainty bounds as a proof-of-concept, using real time snapshots from Trafikverket's API, and training supervised ML models on this data. More specifically, the objective is to compare and evaluate various ML models on data from Trafikverket, and provide an overall accuracy score along with point predictions and prediction intervals for any given delay for any given train and station, evaluating how ML can improve upon and correct operational ETAs using them as input. To achieve this, these research questions must be answered:

RQ1

How accurately can train delays be predicted using supervised ML models trained on real time operational data, and in what ways do the models improve the operational estimation?

RQ2

Which features and modelling approaches contribute most to predictive performance and robustness?

RQ3

How do prediction intervals produced by ML models differ in calibration and width, and what does this imply about their suitability for representing train delay forecast uncertainty?

1.1.1 Proof-of-concept framing

Since the time available for writing this thesis is limited, the data collection necessary for capturing seasonal variation in train delays will not be able to be properly collected. The results of thesis will therefore serve as a proof-of-concept, answering what is possible to achieve with a realised dataset capturing seasonal variation.

1.2 Contributions

In brief, this thesis will contribute to the field of Machine Learning for Forecasting through the following pipeline:

- A temporally structured dataset linking ETA snapshots to realised outcomes.
- A supervised learning framework for delay estimation.
- A comparative evaluation of regression models for real time delay prediction.
- A comparative analysis of prediction-interval construction methods across different regression models, identifying qualitatively distinct calibration failure modes.

1.3 External Partner

This thesis is in collaboration with Framna, a product and consultant agency [10] creating digital products partnering with industry leaders in Sweden and other countries. Their partners include SJ, Systembolaget, Swish, SEB and more. For this thesis, the closest internal collaborator is the SJ team, but no internal code base or data is used.

Chapter 2

Background & Theoretical Framework

The following section provides an understanding of the background problem in depth, how systems currently function and what has yet to be explored. It covers railway delays and their propagation, ETA systems today, ML models and algorithms used in forecasting, as well as related work in this field.

2.1 Railway Delays and Delay Propagation

Defining delays is key to understanding domino effects, multi-factor causes and the applicability for ML. The current landscape allows for different descriptions of delays and delay types, and illustrates the necessity for a detailed definition [43].

2.1.1 Delay definition

Delays are defined by the relationship between originally scheduled departure or arrival time and realised departure or arrival time. A scheduled time of arrival or departure T_S is the time a train is originally set to departure or arrive. A realised time of arrival or departure T_R is the time a train actually does arrive or departure. As a result, a delay D is defined as

$$D = T_R - T_S.$$

The value of D is the delay time. A non-positive value would mean T_R is the same time or earlier than T_S, which indicates no delay.

One way of subcategorising delays is by arrival and departure. Arrival delays happen when a train arrives at a station later than scheduled, and can be caused by e.g. track events [27]. Departure delays occur when a train departs from a station later than scheduled, and can be caused by e.g. station congestion [27]. Arrival delay often propagates into departure delay due to turnaround times and schedule dependencies.

Another way of subcategorising delays is by primary and secondary delays. Primary delays are delays that happen because of one or more external or internal factor, such as weather or rail work. Secondary delays are a direct result of primary delays, where primary delays affect trains to come on e.g. the same route, making following traffic delayed as well [11].

For modelling purposes, a formal threshold must be defined for a time deviation being considered a delay. Many operators (namely Swedish operators) define the threshold for delays as ≥ 5 minutes [7]. Due to the requirement of continuous deviation in regression modelling [34], in this thesis a time deviation will be considered a delay if it is > 0 minutes. In practice this means that if a train arrives at a station later than scheduled, no matter how much later, it will be considered delayed.

Differentiating between primary and secondary delays illustrates that railway delays are not isolated events, but components of a dynamic propagation process within an interconnected network. Defining delay formally and operationally enables deviation to be treated as a measurable quantity rather than a binary event. This perspective is crucial for forecasting, where continuous delay magnitudes, rather than simple on-time classifications, form the basis for predictive modelling.

2.1.2 Network Effects

The railway networks in Sweden are interdependent systems, and there are multiple layers of communication and operations that are fully dependent on each other, making failures in one system critical to others [18, 17]. At a high level, external power feeds the internal power system, supplying both telecom routers and signal stations. When the telecom system fails, it also disrupts signal supervision of many signal stations. In an example from the south of Sweden [18], if the telecom system is disrupted, it disables roughly 45% of signal stations. Combined with one converter failure in another system, affected trains go from about 137 to 242, nearly doubling the negative impact, and illustrating the vulnerability in these systems.

The effects of these tightly coupled systems failing are multiplicative, and escalation scales non-linearly. In the context of delays, such amplification mechanisms contribute directly to propagation, as infrastructure disruptions cascade into scheduling deviations

and secondary delays across the railway network. The non-linear and interdependent nature of these escalations makes delay evolution difficult to predict using simple rule-based approaches, as small disturbances may escalate unpredictably.

2.1.3 Propagation Models

Several delay propagation models have been proposed to capture the dynamics of railway delays. One such model is the one Rob M.P. Goverde presented in 2006 [13]. In his framework, Goverde models the railway system as a network of precedence constraints, meaning a train cannot depart until all predecessor events are satisfied – the train's own journey from the previous station, feeder train connections, and minimum headway clearances. These constraints are encoded in a max-plus algebra framework, where departure times are governed by recursive equations of the form

$$x_i(k) = \max_{j \in P(i)} \left\{ x_j(k) + s_{ij},\ t_i^{\text{sched}} \right\},$$

where the actual time $x_i(k)$ of event i (an arrival or departure at a station) is a result of the predecessor time $x_j(k)$ of event j plus the minimum buffer time s_{ij} between the events, or scheduled time t_i^{sched}, whichever happens later. A delay propagates from j to successor i only if it exceeds s_{ij} built into the timetable between them. If it does, i inherits a delay of $z - s_{ij}$ where z is the delay at j. A single initial delay can thus cascade through the network in a domino effect, with the algorithm tracking all secondary delays that arise [13].

Propagation is both temporal and spatial. Spatially, delays spread across stations as trains move along their routes, and across lines when connections cause waiting. Temporally, delays can persist across multiple timetable periods until a *settling time* is reached – the point at which all delays have been absorbed by available buffer time and slack. Models such as the one Goverde proposes take this into account, allowing delays to be tracked across multiple periods $k = (1, 2, 3, ...)$, computing arrival and departure delays for every train at every station over every subsequent period, giving a complete spatiotemporal picture of delay evolution [13].

While propagation models such as this one capture structural delay dynamics through deterministic timetable constraints, they require detailed infrastructure modelling and predefined buffer times. They do not directly estimate the reliability and uncertainty of intermediate ETA predictions, nor do they capture stochastic process times or aperiodic timetables. This motivates a complementary real time data-driven approach.

2.1.4 Forecasting Challenges

Challenges emerge when forecasting train delays, attempting to predict future outcomes in a dynamic, interdependent system. Contributing environmental factors involve multiple interacting dimensions, temporal dynamics, and over time variance in weather and congestion [23, 37]. Real time updates to schedules and ETAs make rule-based forecasting difficult to apply, and escalation uncertainty poses a challenge handling non-linear relationships.

Initial delay magnitude, buffer times, time of day and location variance are all factors in the operating environment, making interacting dimensions multiple [23]. These variables interact non-linearly. A 5 minute delay at a high-traffic station during peak hours behaves very differently from the same delay at an off-peak rural stop. This indicates that tree-based ensemble ML models might outperform rule-based algorithms and linear models, capturing non-linearity in features and interactions.

An issue with real time prediction is temporal dynamics. Information changes over time, which makes it difficult to forecast delays in a dynamic environment. While predictions made further ahead of an event are operationally more valuable as they give controllers time to reroute trains, adjust timetables, or deploy backup resources – they are also inherently less accurate because greater uncertainty accumulates over longer horizons (the time between moment of prediction and realised arrival or departure) [37]. This means that short-term predictions are often more accurate, but leave little room for meaningful intervention. The prediction quality over time does not increase linearly, however, but rather exponentially [37]. As the horizon shrinks, the prediction quality grows acceleratingly, indicating a need for non-linear evaluation methods. Similarly, dynamic external factors such as weather conditions also contribute significantly to delays [27], and their inherent unpredictability makes long-horizon forecasting particularly challenging.

These challenges highlight the potential difficulty of rule-based and linear prediction models in a landscape where dynamic, non-stationary data changes in real time. Capturing changes over time, not just in environmental features such as weather, congestion and time-of-day, but in delay predictions themselves could require models that can adapt to evolving conditions rather than assume a stable underlying process. As prediction horizons extend and uncertainty grows across stations and train interactions, the gap between a model's historical calibration and the current operational reality widens. This raises a fundamental question: not just *what* a prediction is, but *how much it can be trusted* at any given moment. Addressing this could require a shift toward forecasting frameworks that are explicitly uncertainty-aware, capable of quanti-

fying prediction confidence in real time as conditions evolve. However, whether such complexity is necessary depends on the empirical strength of non-stationarity relative to stable propagation patterns.

2.2 ETA Systems and Trafikverket Open API

The public available data concerning railway information in Sweden comes from Trafikverket's open API [40], where information can be found such as arrival/departure times, train operators, station locations and, importantly, an ETA.

The majority of applicable delay information comes from the end point "*TrainAnnouncement*", where each GET-request returns all updates from the last 36 hours for all stations and trains. Train information updates on ETA revision. Essentially, one data object contains information about one train at one station – advertised arrival time (ATA), station signature, train number, train operator, from where the train is coming, to where the train is eventually going, stations on the route, and an ETA. The ETA deviates from the ATA in definition, providing an estimation given current circumstances of when a train will actually arrive/departure, while the ATA only provides the arrival/departure time originally announced. The ATA contains the same value no matter when the API call is made, while the ETA value is subject to change over time. Hence, calling the API at regular intervals provides updated ETA times for the same train at the same station, giving the data a unique real time element to it. When a train has arrived/departed, a new value can be fetched from the API – the realised time of arrival/departure (RTA) – which contains the time a train actually arrived/departed from a station.

Comparing realised time to the ETA value gives an estimation error, which serves as an answer to the question "*how many minutes off was the ETA?*". Challenges arise, however, when real time uncertainty factors in. Accuracy confidence of the ETA is not included in the API [40], giving no projection for possible cascading or propagation risks. The lack of confidence estimates calls for exploration, both in reliability and communication to end users.

2.3 Machine Learning for Forecasting

This section covers ML for forecasting, and gives a background to supervised learning, candidate model families, loss functions, hyperparameter optimisation, challenges, feature importance and prediction intervals.

2.3.1 Supervised Learning for Regression

In ML, supervised learning means that a model can compare its results to a ground truth, meaning it can access the actual answer of a prediction [5]. In practice, this means a model's training samples, each consisting of one ETA snapshot at time T for train X and station Y, have both input features and output labels. The input features are what is available at snapshot time – current delay, time of day, station, train operator and so on. The output labels are the realised outcomes, i.e. RTA − ETA at snapshot time. The RTA is the ground truth, meaning the model can check this value after prediction and adjust accordingly, making it supervised [5].

ML models produce outputs in one of two forms: continuous numerical predictions via regression, or discrete category assignments via classification [5]. Regression models output a real-valued number, for example *"this ETA will be 4.2 minutes off"*. This makes them naturally suited to answering RQ1 (see 1.1). Prediction quality can then be quantified using standard regression metrics, such as: *MAE*, *RMSE* and R^2 (see 2.3.3) [47]. The main downside of using regression models lies in communication, where continuous values can be difficult to interpret [30]. This is where the latter category, classification, is more apt.

Classification models assign inputs to discrete categories rather than producing continuous values [5] – either binary, such as *"will this ETA deviate by more than five minutes?"*, or multi-class, such as *"low / medium / high deviation risk"*. While classification output is more immediately interpretable to end users, it sacrifices the continuous precision that deviation modelling requires for RQ1 (see 1.1). Classification performance is typically evaluated using *accuracy*, *F1 score* and *AUC-ROC*, which are further explained in Appendix C [49]. The same raw label (RTA − ETA) can serve both tasks – as a continuous target for regression, or binned into risk categories for classification. However, since interpretability also is achievable through deriving classes from regression outputs, only regression is explored in this thesis.

Labels for features are constructed after-the-fact, when joining ETA snapshots with RTAs from the API.

2.3.2 Candidate Model Families

There are many ML models and algorithms that can perform regression and classification tasks. Exploring how these work and in what area they can be used is key to finding the most suitable ones for this problem. This section will explore *linear regression models*, *tree-based ensembles* and *neural networks* as potential candidates.

Linear regression models

Linear regression models are simple but strong models that are used in order to learn a dependent variable (target) as a linear function of one or more independent variables (features) [47]. The target is drawn as a straight line that best fits the data. This equation can be written as

$$y = \beta_0 + \sum_{i=1}^{n} \beta_i x_i + \epsilon,$$

where the target y is a function of the sum of n features x_i weighted by β_i, and some error ϵ. Fitting the model here is a game of finding the best weights β_i for the features used, in order to get the most accurate output for future features. In order to do that, there are a number of algorithms that exist for this purpose. A popular one is *gradient descent*, an iterative algorithm that iteratively adjusts the model coefficients in the direction that reduces error [47]. To measure the current error in the model, a loss function (see 2.3.3) is used. Gradient descent then updates a coefficient to find the value where the error is the least. The formula can be written as

$$\beta \leftarrow \beta - \alpha f'(\beta),$$

where the weight β updates based on the derivative of the loss function for the current weight $f'(\beta)$ and a small constant step size α, which serves as the learning rate. If the derivative is positive at the current weight, the error will increase with the weight. This means the weight must decrease. If the derivative instead is negative, the error will decrease, which means the weight must increase. This relationship allows the derivative function to be subtracted from the current weight for the update, which moves the weight toward the minimum error regardless of which side of it the current value sits on. A challenge with the formula is choosing the optimal learning rate [1]. A step size too small would mean the gradient descent takes unnecessarily long to find optimal weights. On the other hand, a step size too big would mean the gradient descent could leap over the minimum point of error and overshoot, and an optimal weight is never found. Finding a suitable learning rate is therefore key when constructing the gradient descent algorithm.

There are two main implementations of linear regression models – *Ridge* and *Lasso* – both attempting to reduce *overfitting* (when the model fits the training data too well resulting in inability to generalise to new data) by penalising large weights to the loss function [28, 45]. The former, *Ridge*, does this by squaring the weights, meaning

large weights gets a proportionally larger error than small weights do [28]. This allows gradient descent to steer away from solutions with extreme weights – even if those larger weights would have fit the training data slightly better. The latter, *Lasso*, instead penalises large weights by taking the absolute value of the weights, meaning the derivative of the penalty is constant [45]. This means very small weights will eventually reach 0, since they cannot be contributing much to reduce error. Larger weights instead will be punished until they reach a point where their contribution to error reduction outweighs their contribution to overfitting.

Linear regression models assume the relationship between features and target to be a straight line [47]. This means the effect of each feature on the output is constant and independent. The model is therefore easy to interpret and fits well onto data that does have a linear relationship – for example predicting tree height based on radius. Challenges arise, however, when feature data is not linearly related to target values and complex dynamics hinder interpretability [30]. Furthermore, the interactions between features are invisible to the model. The weight β_i only captures the isolated contribution of feature x_i. If two features x_i and x_j affect each other, β_i cannot capture this. The theoretical complexity of delay dynamics (see 2.1.4) suggests that linear models may struggle in this domain. However, whether these non-linearities are systematic enough to be captured by training data is an empirical question. If non-linear effects manifest primarily as stochastic noise rather than learnable patterns, linear models may prove more robust than flexible alternatives by avoiding overfitting.

Regardless of their potential limitations, the simplicity and interpretability of linear models are valuable, precisely because they give a clear reference point as to a model's performance. If a more complex model cannot outperform linear regression, the added complexity is not justified, meaning linear models can set the performance floor for other models. β_i coefficients are directly interpretable, and each weight gives an exact answer to how much a feature is contributing, which is useful for understanding the problem even if the model itself is not the final choice. Conversely, if linear models succeed despite theoretical non-linearity, this reveals that the predictable component of the system is fundamentally linear.

Tree-based ensembles

Tree-based ensembles, or decision tree groupings, are supervised models that are suitable for handling complexity and non-linearity [51]. The simplest versions of these models are single *decision trees*. Decision trees are popular for their interpretability and application range, and can be used in both regression and classification [51]. The core concept is that a tree consists of a series of decisions, where each *decision node*

splits the tree in two or more new branches. The branches that follow too have decision nodes, eventually landing in prediction outputs known as *leaf nodes* [51]. A visual representation of a decision tree can be found in Figure 2.1.

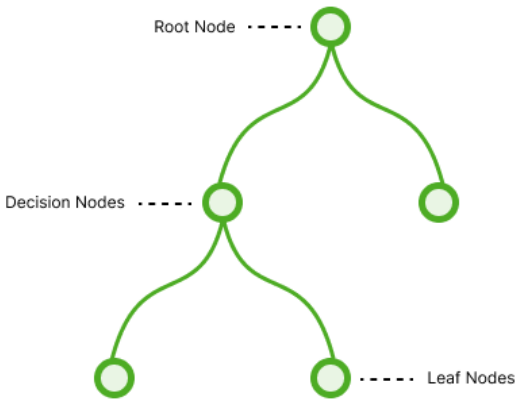


Figure 2.1: Example of a decision tree structure.

The model then goes from the *root node* and downwards, making a decision at each node, until reaching a leaf node, whereupon an output is decided. The decisions at each node can represent values of different features, for example current ETA, station signature or weather conditions. A node then asks for example, "*is the temperature above 0°C?*", upon which the answer leads the model down a certain branch [51]. A simplified visual representation of decision nodes can be found in Figure 2.2.

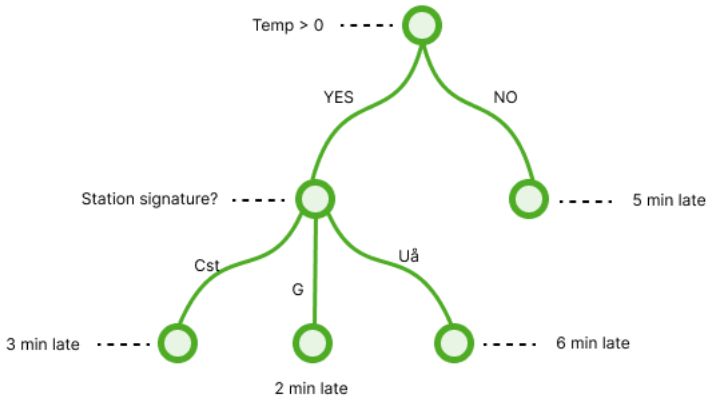


Figure 2.2: Example of decision nodes in a decision tree.

Finding where to partition the data is key for decision trees to function properly. In order to find these sequential rules, *entropy* needs to be understood. Entropy is a measurement of the *impurity* of a set of labels, answering how mixed classes are at a given node [51]. The formula can be written as

$$H(S) = -\sum_{i=1}^{n} p_i \log_2(p_i)$$

where, given a set of labels S that will occur with probabilities $\{p_1, p_2, ..., p_n\}$, the total entropy H is given by the negative sum of all probabilities p_i and a *surprise factor* $\log_2(p_i)$ capturing the asymmetry found in probability information theory – meaning rare outcomes contribute disproportionally more surprise than common ones. H measures the impurity of S as the expected surprise across all classes. This means H is 0 if S only has one possibility, in which one p_i is 1 but all others are 0, meaning entropy disappears when no uncertainty is present. The maximum value of H, $\log_2(n)$, is given when all p_i are equal in which case all outcomes are equally likely [51].

With the use of entropy, data can then be partitioned optimally by calculating *information gain* [51]. The core idea is to be able to build the entire decision tree by choosing node partitions that maximally reduce entropy. The algorithm used to do this is called *ID3*, and is a recursive function that starts from the root node and iterates the tree greedily, with a top-down approach [51]. At each depth of the tree, the difference in entropy is calculated. The algorithm can be written as

$$\Delta IG = H_p - \frac{1}{N_p}\sum_{i=1}^{c} N_i H_i$$

where the information gain ΔIG for a chosen partition is given by the difference of the entropy of the parent node H_p and the sum of entropies of all consecutive children nodes $\sum_{i=1}^{c} N_i H_i$, where the entropy of a child node H_i times the number of data points in that child node N_i gives a weighted sum that is then divided by the number of data points in the parent node N_p. This means that a weighted average child node entropy is calculated. Subtracting this entropy from the parent node's entropy gives a difference in impurity after a partition. The ID3 algorithm does this for every possible feature and cut-off value, meaning all possible partitions are taken into account. From there, the partition with the most information gain is chosen as a decision node.

When no entropy can be reduced from further partitions, a leaf node is set. This leaf node contains the label of the data points, either as the most common class for classification, or the average value of the data for regression [51].

Practical challenges arise in the application of ID3. Mainly, the algorithm is prone to overfitting [51]. Since it continues partitioning until all leaf nodes are pure, the resulting tree can become arbitrarily deep, and memorise training data rather than learn generalisable patterns. This is particularly problematic in noisy domains, where two samples with identical features may have different labels due to unmeasured factors.

To address this, tree depth is typically constrained through *stopping criteria* or *pruning* [51]. Stopping criteria pauses partitioning early based on thresholds such as minimum samples per node, maximum tree depth, or minimum information gain required to justify a partition. Pruning instead builds the full tree, upon which it removes branches that do not improve performance on a validation dataset [51]. Modern implementations of the ID3 algorithm incorporate these mechanisms by default [31].

Another problem that is an intrinsic characteristic of decision trees is their instability [51]. They are very sensitive to changes in data. A minor tweak of the training data can generate a very different decision tree. This means that if a dataset is just slightly different from another, their corresponding decision trees can contrast entirely.

While instability is an inherent issue with single decision trees, it can be exploited as a strength through *ensemble methods* [51]. Rather than relying on a single unreliable tree, ensemble methods train multiple trees on variations of the data and aggregate their predictions. The individual trees may differ dramatically from one another due to instability, but their collective prediction tends to be far more robust than any single tree [22]. This works because the trees' errors differ. If each tree overfits to different noise patterns in its training subset, their mistakes largely cancel out when predictions are combined. Two prominent ensemble methods use this principle in different ways – *Random Forests* and *Gradient Boosting* [52, 46].

Random Forest is an ensemble method that produces aggregated predictions using the predictions of several decision trees [52]. Predictions from the model are made by majority vote, meaning the label that most trees predict will be the output is chosen as the prediction. *Condorcet's Jury Theorem* [6] states that if any and all people are more correct than chance, increasing the number of people in a vote increases the accuracy of the majority – meaning that the probability that the majority is correct increases along with the addition of accurate participants. In this case, it means that as long as individual trees achieve accuracy above random guessing, adding more trees improves the Random Forest's overall performance. An issue with the theorem is that accuracy does not increase if all trees make the same prediction [6]. The theorem assumes independency among voters, meaning if all trees have the same information and reason identically, they are not independent voters but one voter counted multiple times.

To solve the independency issue, trees in a Random Forest must differ in reasoning. To do that, they must have differing subsets of the training data. A method for this is called *bagging*, and it works by randomly taking samples from the training dataset and putting them into subsets [16]. While data points can be included in multiple

training subsets, they can also be included multiple times into the same subset. Bagging therefore makes it very unlikely that two subsets are the same. When then building trees from the subsets, at each partition the features also get pooled into subsets, meaning the features considered for a decision node differ at each partition [16]. This method, called feature randomness, forces trees to consider different features, even weak ones. All of this is a means to the end of making trees independent, allowing the model to improve its accuracy.

Gradient Boosting is another ensemble method that, instead of building parallel trees and letting the majority vote decide, builds trees sequentially where each new tree focuses on correcting the mistakes of the ensemble so far [46]. Instead of getting a majority prediction, Gradient Boosting investigates what errors remain and how the next tree can correct them. The model starts with the simplest prediction, predicting the mean of all labels. It then calculates the current error via a loss function (see 2.3.3) and trains a decision tree to predict those errors. This tree's predictions are then added to the ensemble with a small weight. The process repeats iteratively, each new tree is trained on the error remaining after all previous trees, gradually refining the ensemble's accuracy. The final prediction formula can be written as

$$\hat{y} = f_0 + \alpha \sum_{i=1}^{n} h_i(x),$$

where the prediction $\hat{y}$ is given by the initial baseline prediction f_0 (the mean of training labels) and the weighted sum of all tree predictions. Each $h_i(x)$ is a decision tree that takes input features x and outputs an error correction. The learning rate α (a small constant) dampens each tree's contribution to prevent overfitting, functioning similarly to the step size in gradient descent (see 2.3.2). A simplified example of the gradient boosting process can be found in Appendix B.

An issue with sequential tree building is overfitting. Because each tree targets the ensemble's current errors, the model risks memorising the exact data rather than learning true patterns, particularly as the number of trees grows [2]. Unlike Random Forest, where independent trees provide robustness through diversity, Gradient Boosting's trees are highly dependent on one another. If early trees overfit, later trees further the problem by correcting errors that were never real patterns to begin with.

To address this, Gradient Boosting employs regularisation techniques that constrain each tree's influence. The learning rate α is the primary control – smaller values mean each tree contributes less, forcing the model to build more trees and learn patterns gradually rather than fitting noise aggressively [46]. Additional regularisation comes

from limiting tree complexity. Restricting maximum depth, requiring a minimum number of samples per leaf, or subsampling the training data for each tree (similar to bagging) [46] all contribute to dampen overfitting. These constraints prevent individual trees from becoming too specialised, ensuring the ensemble remains generalisable.

Modern implementations of Gradient Boosting, particularly *XGBoost* and *LightGBM*, extend the core algorithm with optimisations for speed and accuracy [46, 2]. XGBoost (Extreme Gradient Boosting) adds advanced regularisation terms directly into its loss function, penalising both the number of leaves and large leaf weights, giving finer control over tree complexity than depth limits alone [46]. LightGBM (Light Gradient Boosting Machine) instead prioritises computational efficiency through a leaf-wise growth strategy. Rather than growing trees level-by-level, it always splits the leaf with the highest potential error reduction, resulting in deeper but more focused trees [2]. This allows LightGBM to handle large datasets with many features efficiently. Both implementations support categorical features natively, preserving cardinality information that is lost in simpler approaches [2].

Neural networks

Neural networks represent another class of supervised models capable of capturing complex, non-linear relationships through layers of interconnected nodes [48]. Each layer transforms input features through weighted connections and activation functions, allowing the network to learn hierarchical representations of the data. Deep networks with many layers can theoretically approximate any function, making them powerful for tasks where feature interactions are unknown or highly abstract [48]. This does, however, come at a cost. Neural networks are *black boxes*, meaning interpretability of the model's results are difficult. Understanding why a neural network makes a particular prediction requires advanced analysis tools rather than direct inspection of learned parameters [48]. They also demand larger datasets to train effectively, are sensitive to hyperparameter choices, and require significantly more computational resources than linear and tree-based methods [48]. For tabular data such as the data in this thesis, tree-based ensembles often outperform neural networks while remaining simpler to train and explain [2, 48]. Given the thesis objective of producing interpretable reliability predictions (RQ2, see 1.1), linear and tree-based methods may be preferred and prioritised over neural network approaches. Neural Networks will therefore not be further explored in this thesis.

2.3.3 Loss Functions

When training an ML model, it makes predictions that will be wrong to some degree. To measure exactly how wrong these predictions are, *loss functions* are used [47]. There are a number of different ways to do this, and the functions differentiate between regression and classification. Technically, there is a distinction between loss functions and *evaluation metrics*, the former being used during training and the latter being used after prediction [42] – however, for this thesis the term loss function will be used as an umbrella term for both, in pursuit of brevity.

When training a regression model, a popular loss function is *MAE* (Mean Absolute Error). This function summarises a model's overall error across all predictions, with the error unit also being the same as the target unit [44]. The formula can be written as

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|,$$

where the MAE is the average of the sum of all absolute differences $|y_i - \hat{y}_i|$ between true values y_i and predicted values $\hat{y}_i$ for n predictions. This means errors are shown as absolute values where, in this case, the function can give an answer to how many minutes off the model predicts on average. One thing to note with MAE is that errors contribute linearly to the average, meaning a 10 minute delay contributes twice as much to the average as a 5 minute delay. This makes the function less sensitive to outliers such as rare, extreme delays [44].

Another popular loss function used in regression model training is *RMSE* (Root Mean Squared Error). This function, similarly to MAE, summarises overall error but differs in how sensitive it is to outliers [21]. In this case, this means heavy delays contribute quadratically instead of linearly to average score. The formula can be written as

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2},$$

where the RMSE is the root average of the sum of of all differences $(y_i - \hat{y}_i)$ squared. This means errors instead contribute quadratically to the average, such that a 10 minute delay contributes four times as much to the average as a 5 minute delay. The function is therefore less robust to outliers - which can reflect the reality of rare, extreme delays but can also skew the model's accuracy. The square root of the result of this function allows the average error to be shown in the same unit as the target unit, in

this case minutes [21]. In practice, both MAE and RMSE are often reported together, complementing each other well to handle both the reality of outliers and the robustness needed for accuracy [44, 21].

Evaluating how well a regression model fits a dataset is called *goodness-of-fit* measures, and a common example of one is R^2. This loss function, instead of measuring the average error, tells you how much of the target variance the model explains [47]. This score is not in the same unit as the target unit, but is instead a relative measure between 0 and 1, where a score of 1 means the model explains all variance and makes perfect predictions, and a score of 0 means the model performs no better than predicting the mean. The formula can be written as

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y}_i)^2},$$

where R^2 is essentially a fraction of the numerator total squared error of the model $\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$ and the denominator total squared error of just predicting the mean $\sum_{i=1}^{n}(y_i - \bar{y}_i)^2$. The fraction describes how much error remains compared to this baseline, then subtracts this from 1 in order for a higher score to be better. The R^2 score essentially reveals how much the model reduces the error compared to the error from the mean target values. If the model reduces error by 80%, then $R^2 = 1 - 0.20 = 0.80$ [47]. R^2 is not a replacement for RAE or RMSE, since it does not summarise overall error, but rather a complement to the two.

2.3.4 Hyperparameter Optimisation

Hyperparameters are configuration choices made before training begins that control how a model learns, but are not learned from the data itself. For tree-based ensembles, these include tree depth (how many partitions are allowed), learning rate (how aggressively gradient boosting corrects errors), and number of estimators (how many trees to build). Unlike model parameters such as β_i in linear regression, which are optimised during training via gradient descent, hyperparameters must be chosen in advance, and sub-optimal choices can severely limit model performance. If the learning rate is too high, for example, the model oscillates, skipping over the lowest error point, over-correcting and never converging. If it is too low, training slows down significantly, needing more trees and making it difficult to gain accuracy. If tree depth is too shallow, complex and dynamic patterns in the data is difficult to capture. If it is too deep, the model will overfit the data, capturing error's that are not general. The mere sensitivity of hyperparameter choices calls for optimisation, balancing robustness and error reduction with overfitting and performance. Common strategies for hyperparameter optimisation

include *Grid Search*, *Random Search* and *Bayesian Optimisation* [3].

Grid Search is an exhaustive searching method that attempts to find optimal hyperparameter values by training the model for every combination of two parameters. It does this by creating a two-dimensional grid of potential hyperparameter values, training and evaluating the model on each combination, then selecting the combination that yields the highest evaluation score. An example of a visualisation of this grid can be found below in Figure 2.3.

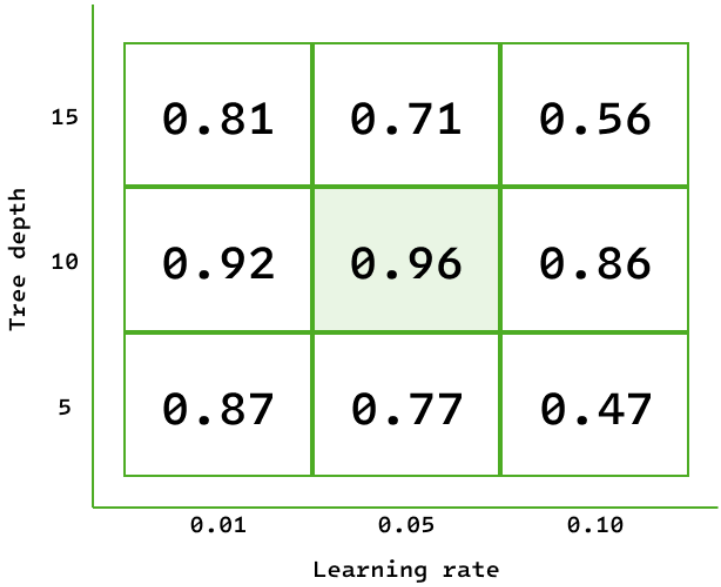


Figure 2.3: Grid search example with two hyperparameters.

In the example in Figure 2.3, two hyperparameters were searched, and the combination that yielded the highest evaluation score was chosen (learning rate = 0.05, tree depth = 10). The caveat with Grid Search is that it runs the ML model on every single combination of hyperparameters. If there are three potential values for each hyperparameter, and there are three hyperparameters, that yields 27 models that will be trained and evaluated. This may be fine for an effective model, but if a fourth hyperparameter is added (also with three options), there are now 81 models. If instead only three hyperparameters are chosen, but each have 5 options, there are 125 models that will be trained and evaluated. The computational cost using Grid Search increases exponentially with each added hyperparameter and potential value. To mitigate these costs, an evaluation score threshold can be set to stop the search when reaching a certain score. Another hyperparameter optimisation approach that attempts to solve the computational cost issue is Random Search.

Random Search functions somewhat similarly to Grid Search. It creates a grid of

potential hyperparameter values, but instead of testing each value, it randomly selects values to evaluate in the given span [3]. This way, a number of times to run can be set, eliminating the additional computational cost from adding more hyperparameters. An example of a visualisation of this grid can be found below in Figure 2.4.

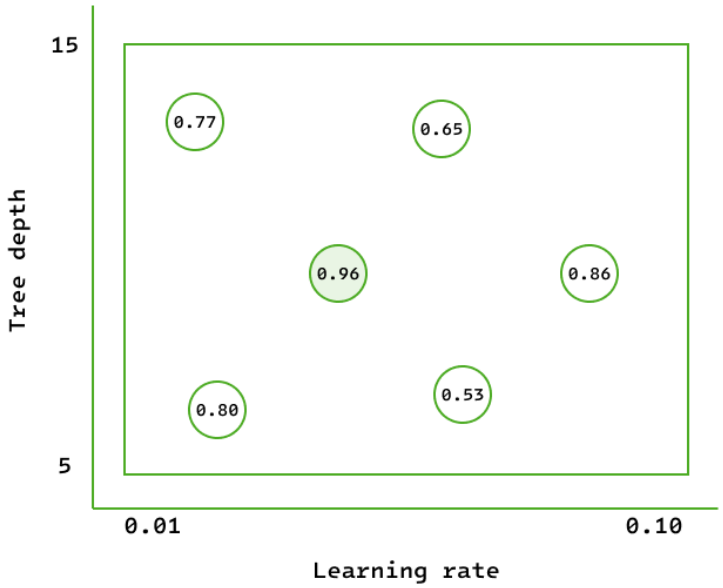


Figure 2.4: Random Search example with two hyperparameters.

In the example in Figure 2.4, two hyperparameters were searched, and the combination that yielded the highest evaluation score was chosen. As stated, a search count limit can be set to mitigate computational costs. In the example above, a limit of 6 was set to stop the search. In Grid Search, combinations are fixed and dependent on the grid definition itself. While this may be preferred if the fixed size is affordable, Random Search allows for choosing the budget, making it more flexible. Another reason Random Search often outperforms Grid Search is its likelihood of optimal outcome [3]. While Grid Search searches through specified parameter values, Random Search can search values that are not explicitly specified. This means it may find the optimal combination, or at least be closer to it, even if it is not explicitly stated as an option.

An issue with both Grid Search and Random Search is that they can suffer from inefficiency due to blindly trying many combinations, without evaluating their usefulness during the search [3]. To address this issue, Bayesian Optimisation approaches the search by learning from past results [36]. It functions similarly to Grid Search and Random Search in the way that it builds a grid of two hyperparameter combinations. It then starts by conducting a few random evaluations and recording their scores. It then builds a new *surrogate model* that predicts score, given hyperparameters. Importantly, the model has uncertainty estimates, meaning it knows where it is confident and

where it is uncertain. For example, if the random evaluations gave a high score for a learning rate of 0.05 then that parameter value is a high confidence area, whereas if a learning rate of 0.02 was not evaluated then that parameter value is a high uncertainty area. The surrogate model therefore gives two pieces of information – exploitation (predicted score) and exploration (uncertainty) [36]. It balances these to by partly wanting to try combinations near known well-performing regions, and partly wanting to try combinations in uncertain regions. The surrogate then lands on a combination to try, trains the ML model, adds the score to the set of combinations, and then repeats the whole process. An example of a visualisation of this grid can be found below in Figure 2.5.

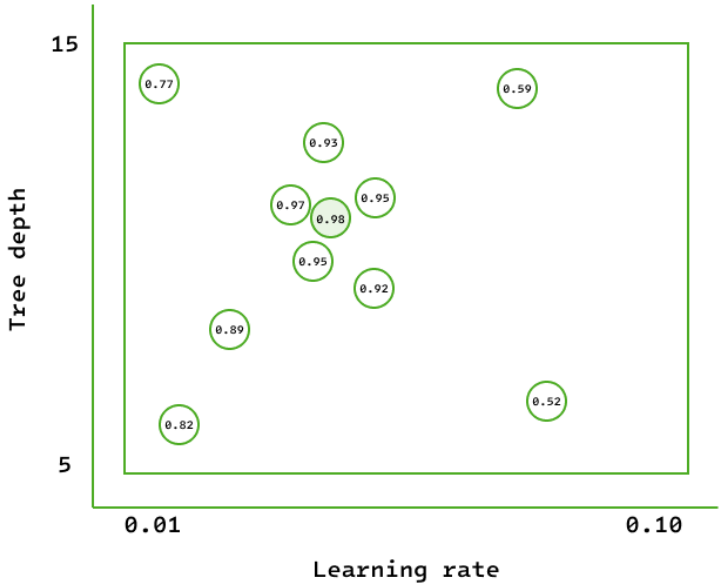


Figure 2.5: Bayesian Optimisation example with two hyperparameters.

In the example in Figure 2.5, two hyperparameters were searched, and the combination that yielded the highest evaluation score was chosen. By learning from previous scores, Bayesian Optimisation can narrow down and focus on regions with good results [36]. This way, it is computationally more efficient than Grid Search, and is more likely to find well performing combinations on fewer iterations than Random Search. The trade-off here is that Bayesian Optimisation is more complex and heavy to implement than the others, making it sub-optimal for small scale data and parameters, but does save time when each evaluation is expensive.

The search methods described above need to find the hyperparameters that yield the best fitted models. However, if the parameters during searched are only fitted on training data they risk overfitting, and as a consequence, the search methods would favour overfitted models. The methods therefore use not only training error, but

validation error [3]. Models train on a subset of the data, almost always a majority. They then validate what they have learned on the rest – validation data. The issue with a single split of training and validation data is that chance becomes a factor. The validation data may contain mostly rare outliers making the model seem to perform worse than it does, or it contains mostly typical outcomes, yielding an accurate performance score. This means the score of a hyperparameter combination could come down to chance in how the training/validation split happened. To solve this issue, *cross-validation* is used [14].

Cross-validation reduces variance in the performance score by averaging across multiple training/validation splits. A common way of doing this is *k-fold CV*, which performs multiple different splits called *folds*, runs the model on each fold, then averages the scores [14]. This yields a much more stable and reliable score. Cross-validation is especially vital for smaller datasets where extreme outliers can heavily affect performance scores.

2.3.5 Challenges

Several challenges emerge using ML for forecasting train delays. Firstly, since all data is temporal (ordered by time), cross-validation becomes difficult. Using common k-fold CV makes the model perform deceptively good, where random training/validation splits in the data almost certainly leads to training on data further in time than the validation data. This means future data informs past predictions, meaning the model is able to see into the future to fit the data better. This is a large overfitting issue, not to mention how it does not reflect the real world in the slightest. The solution is to use another type of cross-validation – *time-series CV* [15]. Instead of randomising splits, time-series CV makes sure data is split by time. It does this and still averages multiple folds using increasing temporal splits. The first fold has a smaller, early training set and a larger, late validation set. As the number of folds increments, the split happens at a later time, meaning the training set size increases and validation set size decreases. It then averages the scores yielded similarly to k-fold CV.

Another challenge that comes with ML for delay forecasting is non-stationarity. As explained in Section 2.1.4, temporal and railway dynamics make it difficult for a rule-based approach to capture the landscape. This challenge can be found in Machine Learning as well, not as a breaking limitation, but rather as a concern regarding unpredictability. Delay patterns shift with season, timetables change and unforeseen incidents occur sparsely. There are delay factors and features that cannot be caught by a model, and models will never account for 100% of them in an environment this dynamic.

There is, in the railway landscape, an imbalance between on-time and delayed trains. Most trains are on time (as introduced in Chapter 1), and extreme delays are very rare. While accounting for a very small portion of the entire dataset, heavy delays are important nonetheless. It is perhaps during these delays that correct estimates hold the highest value. This creates a tension, where optimising for overall RMSE (see 2.3.3) may yield a model that predicts the common case (near-zero deviation) accurately while failing on the high-stakes outliers that matter most for passenger communication and resource allocation.

For answering RQ2 (see 1.1), it is important to understand feature contributions to model results. A challenge with this is when features correlate. Many railway features are collinear, meaning they indicate somewhat the same thing. This means ML models will have a difficult time distinguishing which feature contribute most, and interpreting becomes unreliable [38]. Specifically, weights of features may become interchangeable if they relay the same information.

A final challenge with ML for forecasting is the horizon dependency. As specified in Section 2.1.4, prediction accuracy generally degrades with the prediction horizon [37]. This is true when using ML models as well, and it means that predictions from a model will be more accurate the closer current time is from the ETA. When that horizon is wider accuracy decreases, which could prove a problem for extreme delay prediction.

2.3.6 Feature Importance and Prediction Uncertainty as Interpretability

To define interpretability in the context of ML, a distinction can be made between *global interpretability* and *local interpretability*. The former is understanding overall model behaviour – feature importance, overall accuracy, scoring metrics and all other interpretable results given by the model as a whole. The latter is understanding individual predictions made by the model – what features and metrics influenced that specific prediction, and how reliable it is. This distinction matters because users experience predictions one at a time, benefitting from local interpretability, while system designers and operators need to validate overall logic, benefitting from global interpretability.

In the context of this thesis, RQ2 and RQ3 (see Section 1.1) ask what features and modelling choices contribute most to performance, as well as how prediction intervals expose forecast uncertainty. Both are issues of interpretability, which requires transparency. End users are more likely to trust predictions when they understand why a reliability score is low or high [26]. This means that black-box predictions (predictions

without interpretable reasoning) will not satisfy user trust, while interpretable predictions will. End users need simplified explanations for individual predictions to inform travel decisions, and system designers and operators need global feature importance to verify that a model is learning sensible patterns. Interpretability also relates to ethics in communication, where if a prediction for example causes a passenger to miss a connection or abandon a journey, explaining the reasoning behind it often gains trust rather than loses it [26]. Estimation interpretability is therefore a prerequisite for accountability.

Another importance feature of interpretability is that railway delays have known causal factors. Propagation, weather conditions and time-of-day congestion are all well known delay causes. Interpretability allows for validating that a model is learning these domain-grounded patterns. If feature importance scores show that some factor less vital matters more than for example latest known delay, that is a warning sign suggesting that the model is overfitting to noise. Additionally, communication methods need to be grounded in interpretability outputs. In order to explain for example a reliability confidence interval to end users, interpretability is necessary.

Interpretability is therefore vital for communication and trust, and there are ways of gaining it from ML models, mainly through *SHAP* or *permutation importance*.

SHAP Values for Local and Global Explanations

SHAP (SHapley Additive exPlanations) is based on Shapley values from cooperative game theory and aims to answer how much each feature contributes to moving a prediction away from the average prediction [33]. It decomposes a prediction into a sum of feature contributions, so that a prediction can be explained through how much each feature contributed.

SHAP works for all ML tasks and models, allowing usage of the same interpretability method across linear regression, random forests, gradient boosted trees and neural networks. Granting feature contributions for each individual prediction, it is especially suitable for local interpretability [24].

For a given prediction, SHAP computes how much the prediction changes when including or excluding each feature, and when averaging over all possible feature combinations – making it a weighted average of marginal contributions [24]. The output of SHAP algorithms are commonly given in plots. SHAP is therefore useful for RQ2 (see 1.1), granting local explanations and interpretation. While SHAP can be used for global importance, it is not explored in this thesis.

Limitations regarding SHAP are, mainly, computational [33]. It is expensive to run

SHAP algorithms for large datasets. Another limitation is that SHAP values do not show causal effects. While it may appear that they are, SHAP values show correlation within the ML model, not real world causation. It shows how the model learned to associate certain features with certain outcomes, not that they cause the outcomes directly. A final limitation of SHAP is how feature interactions can make interpretation more difficult. While dynamic interactions between features can be explained by SHAP values, that explanation is often complex.

Permutation Importance as a Complementary Metric

A metric that complements the limitations of SHAP is permutation importance. It is a global interpretability method, measuring how much model performance degrade when *shuffling* a feature's values. Permutation importance randomly permutes features and looks for changes in loss function scores [4, 9]. If RMSE (see 2.3.3) doubles after a shuffle, that feature is important, while if RMSE stays roughly the same, the feature contributes little.

Permutation importance works by training and evaluating a model, then for each feature randomly shuffling that feature's values in the validation set [9]. This breaks the relationship between that feature and the target. After evaluating the model again, post-shuffle, importance score is given by subtracting the original evaluation score from the shuffled one. This repeats for every feature so that features can later be ranked by importance.

Permutation importance has some advantages over SHAP [9]. It requires no complex computation since it only trains once. It is conceptually simpler to explain, and it too works for any ML model. Limitations, however, include scale. The metric is only global, meaning it does not give per-prediction explanations. It is also sensitive to feature correlation, meaning if two features are correlated, permuting one might not degrade performance much since the other compensates [38]. Furthermore, permutation importance does not show direction. It answers what features are important, but does not provide explanations for whether high values increase or decrease predicted outcomes. SHAP, on the other hand, shows this via positive and negative contributions. Permutation importance should therefore be used as a complement to SHAP, not a replacement.

Prediction Intervals

In order to answer RQ3 (see Section 1.1), prediction intervals must be explored. They can be defined as a range around a point prediction such that the label for that

prediction falls inside the range with a certain probability [20]. This is in contrast with a point prediction, which gives no information about confidence [20].

To assess whether a prediction interval is reliable, a *nominal coverage* must be defined, which works as the target coverage for the model to hit [32]. The *empirical coverage* is the share of cases where the prediction label falls inside the prediction interval. A well-calibrated interval has empirical coverage close to the nominal coverage. If there is under-coverage, it means the model is over-confident and has too narrow ranges for true values to fall inside them. If there is over-coverage, it means the model's ranges are excessive in width without improving confidence [32].

How prediction intervals are constructed differs by model family. For gradient boosted trees such as LightGBM, loss function based *quantile regression* is used [20, 19]. A separate model is trained with the loss at each desired quantile (upper and lower percentage bounds of the interval). For other ensembles such as Random Forest, *tree-prediction-percentile* methods are used [25]. For each test point, the lower and upper bounds are taken as percentiles across the predictions of the individual trees in the ensemble. Linear models, such as Ridge, instead use *independent quantile fits* [20]. The lower and upper bounds are fitted as independent quantile regression models, with no monotonicity constraint relative to the point estimator. While these models differ in implementation, the outcomes follow the same pattern, with a prediction interval consisting of an upper and lower bound within which the model is some percentage confident the true value will fall.

Intervals fit into interpretability by their widths being per-prediction signals. A narrow interval tells the reader the model is confident, and a wide one tells them it is not. This can therefore be used as further local explanations alongside SHAP. The limitation of prediction intervals is the fact that the bounds most often have to be fitted as separate models from the point prediction. This means a point prediction could fall out of bounds of the interval, and the lower and upper bound can invert. The severity of this issue is method-dependent, but it is still vital to evaluate not only empirical coverage, but crossover occurrences and bound inversions as well.

2.4 Related Work

Train delay forecasting has been approached from two broad directions – *analytical propagation models* and *data-driven ML models*, the former being a deterministic approach that focuses on a known and predetermined environment, and the latter a stochastic method that focuses on dynamic data and trying to find hidden patterns using ML. Recent work has begun moving toward uncertainty-aware and real time

variants of both. This thesis sits between data-driven point prediction and uncertainty quantification, with the distinctive framing of correcting and improving on an operational ETA. Below, previous work in this broad field will be discussed subcategorised by their main contributions.

2.4.1 Analytical Propagation Models

Analytical models treat the railway network as a system of precedence constraints and recursive scheduling equations, propagating an initial disturbance through the timetable deterministically. As the canonical example, thoroughly explained in 2.1.3, Goverde (2006) proposes the max-plus framework, that models railway systems as delays that propagate when they exceed built-in buffer times, cascading both spatially across stations and temporally across timetable periods [13].

The strengths to acknowledge about these deterministic models are their structural fidelity to railway operations, and their interpretability of cause-and-effect relationships. Furthermore, no data collection is required for a analytical propagation model.

The limitations of the models, as they relate to this thesis, include requiring detailed infrastructure modelling and predefined buffer times, which are not exposed by Trafikverket's open API. They also do not natively produce uncertainty estimates around predicted delays, which is vital for interpretation and communication. They assume periodic timetables and deterministic process times, which may not be the case in a highly dynamic environment, as they cannot capture stochastic variation. Lastly, they operate on a train schedule, and not on streaming ETA updates. This means they cannot consume the real time signal that the API from Trafikverket provides.

These limitations motivate complementary data-driven approaches that learn directly from real time operational data, including the ETA updates the operator already publishes.

2.4.2 Data-Driven Delay Prediction

Data-driven approaches treat delay prediction as a supervised learning problem, finding relationships between observable features and realised delay outcomes from historical data. For example, Gao et al. (2023) proposes a deep learning model for short term train delay prediction, that predicts total delay magnitude using a deep neural network [11]. In their case, the prediction target is the delay itself, computed from scratch, and does not include operational forecasts to improve upon. Liu et al. (2023) proposes another model that learns high-speed train delay propagation using casual text information [23]. They engineer features from textual incident reposts, and not

tabular ETA data – an orthogonal direction to this thesis. The paper is useful as a contrast in feature philosophy, where text-derived causal signals differ from numerical ETA evolution. Oneto et al. (2018) researches train delay prediction systems from a big-data perspective [29]. It is the closest methodological cousin to this thesis, using operator data, having a ML pipeline and comparing multiple models. Their prediction target in this paper is also delay magnitude using operational history, but like Gao and Liu does not consume an existing operational forecast as a feature. The paper also leaves exogenous data untouched, leaving a gap for this thesis to explore weather data as features.

The common pattern across the papers above is that the prediction target is delay magnitude, but the operational ETA published by the infrastructure manager is treated as the system's output rather than as an input to the ML model. The model essentially competes with or replaces the operational forecast.

This thesis shares the prediction target, delay magnitude, with the above works, but differs in two ways. Firstly, Trafikverket's operational ETA is used as an input feature, reframing the task from producing a delay forecast from scratch, to correcting and improving upon the existing operational forecast. Secondly, the dataset is snapshot-keyed, capturing how the operational ETA evolves over time rather than collapsing each event into a single observation.

2.4.3 Uncertainty-Aware Delay Forecasting

Uncertainty in delay forecasting matters because a single point prediction does not communicate confidence, which limits both operational decision-making and end-user trust. Despite a large body of stochastic delay prediction work, evaluating the quality of the produced uncertainty has received comparatively little attention. This is the gap the uncertainty-aware ML literature has begun to address. Research on the topic has been done by for example Spanninger (2023) [37]. Spanninger predicts real time arrival delay with associated uncertainty, using a mixture of offline features (day of week, train ID, etc.) and online features (current delay, train position, etc.). To be noted is the absence of an operator-issued ETA as an input, meaning this model competes with a timetable, not with a real time forecast. The associated uncertainty framework is here an uncertainty-aware neural network that simultaneously estimates a point prediction and the parameters of a Gaussian distribution around it, which can be interpreted as a prediction interval. Spanninger evaluates the framework via prediction interval coverage and Likeliness of Realization (LoR) within the predicted distribution, methodologically equivalent to the empirical-coverage evaluation strategy introduced in Section 2.3.6. A major contribution of Spanninger's thesis is showing that predictability

decays exponentially with prediction horizon, with uncertainty dominating as horizon grows.

The similarities between Spanninger's work and this thesis are multiple [37]. Both theses treat delay predictions as a dynamic problem updated as a train progresses, not a one-shot forecast. Both recognise that horizon and uncertainty interact, with Spanninger's exponential decay finding paralleling this thesis' horizon-stratified skill analysis explained further in Section 3.5.5. Additionally, both theses have a methodological alignment on evaluating intervals empirically, not just on point accuracy.

What this thesis does differently than Spanninger's is firstly the model family comparison [37]. Spanninger develops one uncertainty approach and benchmarks its accuracy against six stochastic alternatives, but the uncertainty machinery itself is held fixed. This thesis instead holds the evaluation fixed and varies the model family, exposing how each family's native interval-construction method performs on the same data.

Prior uncertainty-aware delay work, most notably Spanninger [37], commits to one interval-construction method and evaluates its accuracy in depth. This thesis instead compares how three model families' native interval methods perform on the same forecast correction task.

2.4.4 Thesis Positioning

There are a number of distinguishable features of this thesis not found together in prior work. The dataset structure is a snapshot-keyed design that captures how an ETA evolves between issuance and realisation, while most prior work uses static or end-of-event data, thus losing the real time signal entirely.

The prediction target of the thesis is the total delay magnitude as most prior work, but the important distinction is that operational ETAs are used as input in order to improve upon and correct it, not compete against it from scratch. This allows the ML models to align with what the operational system already does well.

There is a comparative uncertainty analysis in this thesis. Three model families are evaluated not only on point-prediction accuracy but on the quality of their native prediction intervals. The findings of the three native methods (results of Section 4.2.2) will be methodological contributions in their own right.

Although the dataset is limited to a single seasonal window (as the proof-of-concept framing describes in Section 1.1.1), the structure and findings will establish a reproducible template for further work using real time operational API's.

Chapter 3

Methodology

In this section, the methodologies chosen to be able to execute the labour of the thesis is described in detail. The section covers data collection, dataset construction, feature choices, model choices, evaluation protocols, ethical considerations and end user communication.

3.1 Data Collection

No historical data was available on Trafikverket's open API. This means only the current state of railway traffic was returned per API call. The data therefore had to be collected through continuous fetches. In detail, two collection programs were built running on a local machine. Scripts were executed using *cron jobs*, which allows scripts to be scheduled and executed at a certain time or at certain time intervals, meaning automatic fetches can be made from the API. The two cron jobs built were a *snapshot collector* and a *label collector*, which collected a number of data variables as either snapshots (feature instances) or labels (targets). A chart explaining variables captured by the scripts can be found below in Table 3.1.

Table 3.1: Variables captured by the collection scripts.

Variable	Description	Captured as
`activity_id`	Announcement ID	Snapshot
`advertised_train_ident`	Train number	Snapshot
`location_signature`	Station code	Snapshot
`activity_type`	Arrival or departure	Snapshot
`advertised_time_at_location`	ATA	Snapshot
`estimated_time_at_location`	ETA	Snapshot
`eta_effective`	ETA or ATA	Snapshot
`cancelled`	Cancellation status	Snapshot
`operator`	Train operator	Snapshot
`time_at_location`	RTA	Label

The snapshot collector ran every 5 minutes. The script queried the Trafikverket API, capturing a window of 30 minutes back to 6 hours ahead from current time, ensuring that upcoming events were observed multiple times before their scheduled occurrence and late system updates were caught. Each snapshot recorded the variables listed in Table 3.1, capturing ATAs and ETAs, station and train identifiers, and cancellation status. Importantly, the collector also calculated an effective ETA (ETA if available, otherwise ATA). The goal of the snapshot collector was to collect important features at regular time intervals, in order to later see how the data evolved over time.

The label collector ran once every day at midnight. The script queried the same API endpoint, and captured a 36 hours back to now window in order to observe the realised arrival or departure times for all events previously fetched by the snapshot collector, including late system updates. The fetch recorded RTA for the announcements captured by the snapshot collector, as well as recorded event ID for later matching.

Storing of the data was done via *SQLite*, an in-process library that allows storing data in a self-contained database as a single file. Data from the snapshot collector and label collector was put into two tables in the database. The database was updated on every run of the scripts.

Because of the limited time frame available to write this thesis, the collection period was from 2026-02-02 to 2026-03-02 (29 days). This was a large limitation, since seasonal variation is an important feature in train delay estimates (which is discussed as a limitation in 5.3.1).

In order to minimise database space and time complexity, some deduplication was made. Since snapshots could be caught multiple times from the snapshot collector, the

script ensured unique indexes for snapshots, essentially meaning no snapshot event could have the same ETA and ID without being considered a duplicate. If the same event was fetched again with unchanged ETA and status, instead a *seen count* was incremented along with an updated time keeping variable. This ensured that only state changes, such as an updated ETA or cancellation status, produced new rows rather than repeated observations of unchanged predictions. The total number of snapshots in the database after collection stopped was 544,064. A schema over how data was collected can be found below in Figure 3.1.

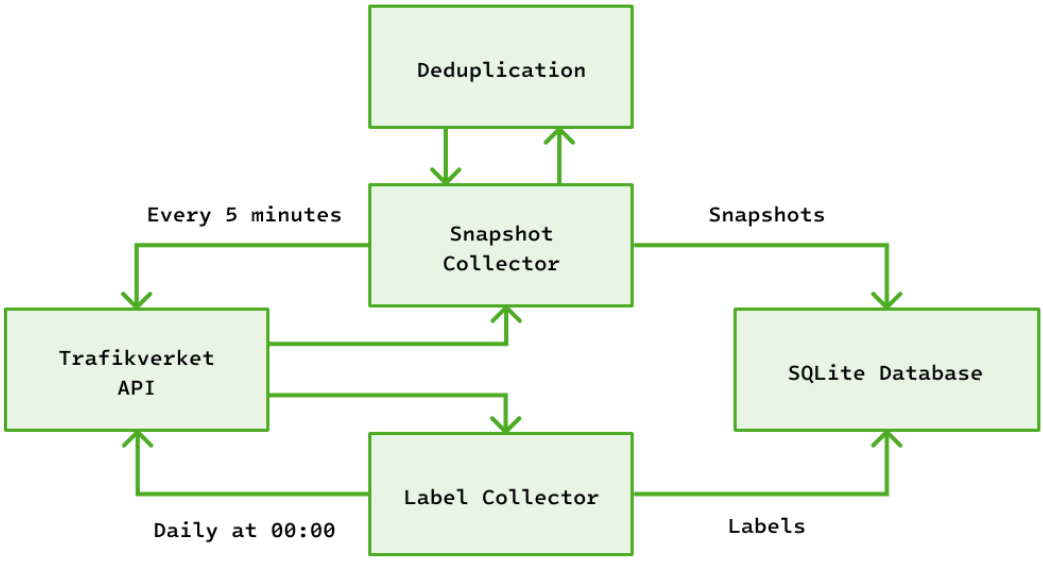


Figure 3.1: Data collection schema.

3.2 Dataset Construction and Labelling

For the dataset construction, the snapshot and label tables were joined together. Since all labels had the same event ID as the snapshot rows for the same activity, matching the pairs allowed for a ground truth outcome of RTAs. Only non-cancelled events where RTA was not null were kept, since cancelled trains and events without an RTA had no usable label. This resulted in 283,014 train events, represented by 456,652 snapshot rows.

From the joined data, the prediction target was given by

$$D_i = T_{R_i} - \hat{T}_{S_i},$$

where the deviation D_i in minutes was given by the difference of the RTA T_{Ri} and the ATA $\hat{T}_{S_i}$. A positive D_i meant the ATA was too optimistic, meaning the train arrived or departed later than advertised. A negative D_i meant the ATA was too pessimistic, and a value of 0 meant it was accurate.

In the final dataset, each row represented one snapshot – a single train at a single station, either an arrival or departure, captured at a specific time. This was the *unit of prediction*, which defines the specific entry for which a prediction is made, and all machine learning methods and decision rules were based off of this unit.

Weather observations were collected separately, from SMHI's open meteorological API. Each train station was matched to the nearest SMHI weather station using Haversine distance [35], producing a station mapping table. Weather parameters such as temperature, precipitation, wind gust and dew point were then joined to events by station and observation time. A table over the SMHI parameters used from their open API can be found in Table A.1.

Before stashing the weather data in the database, each parameter was scanned by how many weather stations had that data point. If less than 70% of stations had the parameter, it was removed from the data entirely. The threshold was chosen because some stations would otherwise produce excessive missing-ness after joining station-to-station, since if a parameter is missing at the nearest station, it cannot be imputed without using a much more distant station. The remaining parameters were put in a table along with another table mapping weather stations to train stations, all put in the SQLite database.

In order to enable a feature capturing delay propagation as described in Section 2.1.3, snapshots were grouped into journeys. The grouping logic worked as follows – all events sharing the same train number were sorted chronologically. They were then split into separate journeys whenever a time gap exceeding 2 hours occurred or the first departure station repeated. The 2 hour mark was chosen since gaps between scheduled events on the same SJ long-distance train number that exceed this gap strongly suggest the train number has been resused for a separate journey. This produced a table of journeys in the SQLite database where each row belonged to an ordered journey, enabling a lag operation that, for any event, could look up the delay at the previous departure stop in the same journey.

For each snapshot, the difference between the effective ETA and the event ATA gave a measure of what Trafikverket's system was predicting at the time of the snapshot. This allowed for featuring the latest updated ETA, which the models then could learn to correct. Furthermore, ETA history was retrieved through matching of event ID's to

previous snapshots of the same event in order to derive previous ETAs, delay changes and number of updates from Trafikverket.

Additionally, a horizon feature was calculated for each snapshot, subtracting the time of fetching from the ATA. This allowed for featuring the current event horizon, which granted both insight into reliability and another correction parameter.

3.3 Features

Below, in Table 3.2, all feature groupings used for training the models are listed.

Table 3.2: Feature groupings.

Group	**Features**
Temporal	`hour`
	`day_of_week`
	`month`
Event type	`is_arrival`
Categorical	`location_signature`
	`operator`
Weather	`temp_instant`
	`temp_avg_daily`
	`precip_15min`
	`precip_daily`
	`wind_gust`
	`dew_point`
Propagation	`prev_dep_delay`
Real Time ETA	`eta_delay_minutes`
	`horizon_minutes`
ETA History	`eta_delay_lag1`
	`eta_delay_lag2`
	`eta_delay_delta`
	`eta_delay_rolling_mean`
	`n_snapshots_seen`
	`minutes_since_last_snapshots`

The temporal features captured time-of-day, weekly and seasonal patterns. These

were categorical but treated as integer features since they still can be treated as such naturally.

The weather features were joined by flooring times and matching to the observation frequency. Cells where values were missing got filled in with column median.

The propagation feature `prev_dep_delay` captured the delay at the most recent departure stop upstream in the same journey. This was a direct operationalisation of the propagation dynamics mentioned in Section 2.1.3, so that if a train was late leaving the previous station, it was likely to arrive late at the next.

The real time ETA feature group calculated estimations from Trafikverket in different ways. The `eta_delay_minutes` feature captured the latest available ETA for an event, provided directly from Trafikverket. The `horizon_minutes` feature captured the current horizon between current time and ATA, as referenced in Section 2.1.4.

The feature group of ETA history captured multiple ETA related data over time. Two lag features were incorporated, capturing the ETA of the prior snapshots leading up to the current. A delta value was also calculated, capturing the changes in delay since the previous snapshot, as well as a mean of the three previous snapshots. Further metadata were also used as features, where the cumulative count of prior snapshots and minutes since the last snapshot was fetched were captured.

Missing values in numerical features were imputed with the training-set median, computed once per feature and applied to both training and validation rows to avoid leakage. Categorical features had no missing values after the cancellation filter. Two features received non-median treatment – `prev_dep_delay` was filled with 0 when the upstream departure had not yet occurred at the snapshot's fetch time (since by definition no delay had been observed), and `latest_eta_delay_minutes` was filled with 0 when no pre-event ETA snapshot existed (the train was considered on-time by default).

3.4 Models

The three ML models used and compared for this thesis were *Ridge*, *Random Forest* and *LightGBM*.

3.4.1 Ridge

Ridge was used as a linear baseline, in order to set the performance floor for the other two models (see 2.3.2). Since a linear model was in theory unlikely to be the

best performer, it held value as a clear reference point as to a model's performance. Furthermore, the model's performance compared to more complex solutions could prove linear relations to be sufficient. The opting of Ridge over Lasso comes from the fact that Lasso drops features that are deemed irrelevant. Since the features in this thesis were deliberate and relatively few, letting them all be present in the baseline allowed for measuring their effect individually, which served better to RQ2 (see 1.1). The categorical features used (see Table 3.2) were not natively handled by Ridge, since the model expects numeric inputs. These features therefore had to be one-hot encoded, meaning the features get split by their available values and therefore instead create a new feature for every value available, either 1 or 0.

3.4.2 LightGBM

LightGBM was chosen as the Gradient Boosting model. The model reduces bias by sequentially fitting each tree to the error of the previous ensemble, directly aligning with the prediction task of correcting Trafikverket's existing ETA. It handles non-linearity and feature interactions natively, and supports categorical features without one-hot encoding. It was preferred over XGBoost for its histogram-based splitting and leaf-wise tree growth, which substantially reduce training time and made hyperparameter tuning feasible within the time budget of the thesis.

3.4.3 Random Forest

Random Forest was chosen as the third model, for its methodological position in-between Ridge and LightGBM. It occupies a middle ground next to the other models as a non-linear, interaction-capable model that learns without LightGBM's sequential bias-correction. It also uses bagging instead of boosting, and later results would therefore show whether sequential error correction matters more than non-linear interaction capabilities. Beyond its comparative role, Random Forest was well-suited to the characteristics of the dataset. Its variance-reduction inductive bias matched the moderate sample size, and dampens the influence of noisy operational feature values arising from faulty system updates and irregular snapshot intervals. It also pairs naturally with permutation importance, providing a methodologically distinct view on feature contribution alongside SHAP and supporting RQ2 (see 1.1).

3.5 Evaluation Protocol

This section covers the protocol for which the models were evaluated with.

3.5.1 Training and Validation Partition

The dataset was, before full scale model training, split into two subsets – a training set and a validation set. This was not a random split, but rather a temporal one – the split was at a fixed ATA date (2026-02-25). All events before the split were training data (348,197 rows), and all after were validation (108,455 rows). This prevented temporal leakage where future information could inform past predictions.

3.5.2 Hyperparameter Optimisation

Hyperparameter optimisation was performed on all models before full-scale training. For Random Forest and LightGBM, Random Search was used for its non-complex approach while still being able to optimise computational efficiency by setting a fixed number of candidates. Both models had their number of possible candidates set to 30. When it came to Ridge, there was only one hyperparameter to tune. Tree depth, learning rate, and other tree-based parameters are non-existent in linear models. This meant Grid Search could be used efficiently for the model, and was therefore chosen with 7 possible candidates.

Cross-validation was performed for all models using time-series CV with 5 folds during hyperparameter tuning, in order to reduce variance in the performance score.

The chosen values for each model are listed below in Tables 3.3, 3.4 and 3.5.

Table 3.3: Random Search: chosen hyperparameter values (LightGBM)

Hyperparameter	**Search range**	**Chosen value**
Number of leaves	[31, 63, 127, 255]	31
Learning rate	[0.01 - 0.20]	0.0148
Minimum samples per leaf	[10, 20, 50, 100]	50
Row sample by tree	[0.5 - 1.0]	0.6224
Feature sample by tree	[0.5 - 1.0]	0.5698
L1 regularisation	$[10^{-4}$ - $10^{1}]$ (log scale)	3.25×10^{-4}
L2 regularisation	$[10^{-4}$ - $10^{1}]$ (log scale)	0.51

Table 3.4: Random Search: chosen hyperparameter values (Random Forest)

Hyperparameter	**Search range**	**Chosen value**
Number of estimators	[100, 200, 300]	300
Maximum depth	[10, 20, 30, 40]	10
Minimum samples per leaf	[2, 4, 8]	8
Maximum row sample by tree	[0.6, 0.7, 0.8, 0.9, 1]	0.6
Maximum feature sample by tree	[sqrt, 0.3, 0.5]	0.3

Table 3.5: Grid Search: chosen hyperparameter values (Ridge)

Hyperparameter	**Search range**	**Chosen value**
Regularisation strength	[10^{-3}, 10^{-2}, 10^{-1}, 1, 10, 100, 1000]	100

3.5.3 Error Metrics

Loss functions, as defined in Section 2.3.3, were used to evaluate error in the models. *MAE*, *RMSE* and R^2 were used. Furthermore, model *skill* was evaluated by comparing the loss metrics to a naive baseline – Trafikverket's ETA. This comparison was made because the job of the models was to correct the estimate, not predict delays from scratch. Skill was calculated by comparing the model's *RMSE* to the *RMSE* of just predicting Trafikverket's ETA, giving a percentage skill increase or decrease for the model.

3.5.4 Feature Importance

In order to answer RQ2 (see 1.1), features were evaluated using SHAP and permutation importance. As complementary methods, the two metrics showed both local and global feature importances. For local explanations, SHAP values were shown for highest prediction, lowest prediction and median prediction. Permutation importance for global explanations ranked features by their impact of overall $RMSE$. Both methods sampled a subset of the original dataset for efficiency.

3.5.5 Result Stratifications

Along with general loss function evaluation, results were stratified by horizon in order to test whether model skill over the operational ETA varies with forecasting horizon. The horizon bins evaluated were 0-15 min, 15-60 min, 1-3 h, and 3-6 h. The loss metric

scores were shown by horizon bin, as well as compared to the baseline, along with the share of snapshots in each bin and the model's bias (average minutes predicted over or under actual delay across all predictions). A positive bias means the model was under-predicting, and a negative the model was over-predicting.

Results were also stratified by delay bins, in order to test whether model skill over the operational ETA varies with delay severity. The results were shown as in the horizon stratification, but here instead categorised into the bins *early* (delay ≤ 0 minutes), *on-time* (delay ≤ 5 minutes), *minor delay* (delay ≤ 15 minutes), *major delay* (delay ≤ 30 minutes), and *severe delay* (delay > 30 minutes).

3.5.6 Prediction Interval

In addition to point predictions, each model produced a lower and upper bound around its prediction, forming a 90% prediction interval. These intervals communicated forecast uncertainty, where a wide interval signalled that the model is unsure, a narrow one that it is confident. These intervals were evaluated by their empirical coverage, meaning the share of events for which the prediction target (RTA) fell within the predicted bounds, and then compared to the nominal 90% target. Since the upper and lower bounds technically were trained as additional stand-alone models (for LightGBM and Ridge), there was no guarantee that the point prediction would fall in-between the bounds, nor that the upper bound would fall above the lower. Thus, crossover checks were made, investigating the share of point predictions fallen out of bounds as well as the number of bound inversions. The prediction intervals were used due to them being a direct communication of confidence and uncertainty, granting end users reliability interpretation, and serving as an answer to RQ3 (see Section 1.1). Output results of the individual point predictions and intervals were then used for plotting.

3.6 Ethics and Data Considerations

All data used were publicly available through open API's from Trafikverket and SMHI. This meant no personal data was collected – for example, no individual passengers or railway workers were identifiable in the dataset. All identification used were station codes and train numbers, which were operational codes and not personally identifiable.

The research did not involve human subjects – data collection was purely observational on public infrastructure data.

3.7 Implementation notes

The following section covers implementation notes briefly, including programming languages, hardware, and runtime environment.

3.7.1 Programming Languages

The main programming language used for training, feature engineering and evaluation was Python 3.9.6. SQL was also used before training to handle and manipulate the SQLite database. Listed in Table A.2 are the Python libraries used and their usage area.

3.7.2 Hardware & Runtime Environment

All models were trained on a single MacBook Pro with an Intel Core i9 processor and 32 GB RAM. Training the LightGBM and Ridge models each took around 15 minutes, while the Random Forest model took about 40 minutes (all including hyperparameter optimisation and prediction interval training).

Chapter 4

Results

The following section describes the results gained from all models trained and evaluated. Results include descriptive statistics, model performance, and error analysis.

4.1 Descriptive Statistics

The descriptive statistics below establish what the validation set looks like, independently of model results. This section covers dataset composition, snapshot density, delay distribution, forecast horizon distribution, and the operational baseline against which models are compared. A summary table consolidates the key statistics.

4.1.1 Dataset composition

The total validation set contains `108,455` snapshots. Of those snapshots, unique activities captured are `69,852`, and each activity contributes between 1 and 20 observations, with most contributing only one. The date range of the validation data is from `2026-02-25` to `2026-03-02`. This is the temporal validation split mentioned in Section 3.5.1, and the training set covers a prior period from `2026-02-02` to `2026-02-24`. The unit of prediction, and therefore unit of analysis is one observed state of one event (one train departure or arrival at one station) at one snapshot time.

4.1.2 Snapshot density

The mean snapshots per activity is `1.55`. The median is `1`. The distribution of rows per activity is plotted below in Figure 4.1.

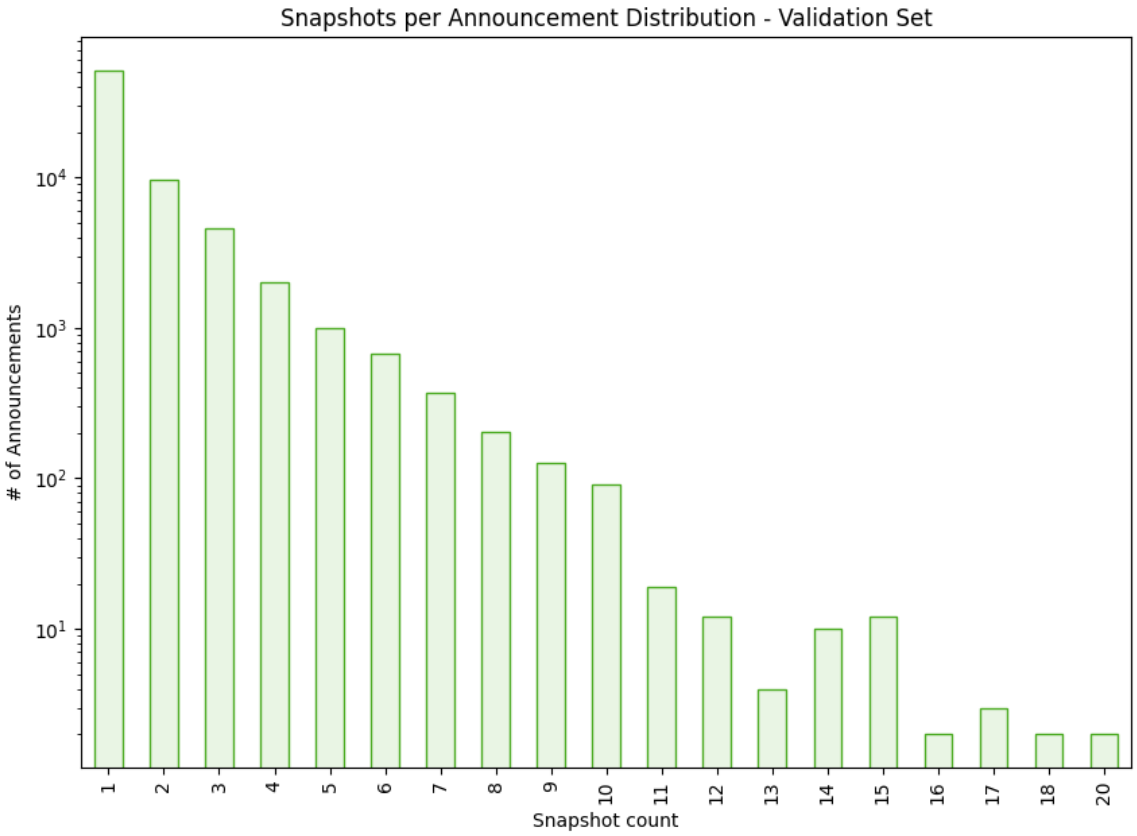


Figure 4.1: Distribution of observed states per activity in the validation set.

The number of activities with a single observation is a clear majority in the validation set, at about 73%. Single observation events are those whose published ETA never changed throughout the observation window. Multi-row activities captured ETA revisions during collection. Since most train events reached their advertised time without ETA revisions, this statistic has a heavily left-skewed peak.

4.1.3 Actual delay distribution

The distribution of actual delays is plotted below in Figure 4.2.

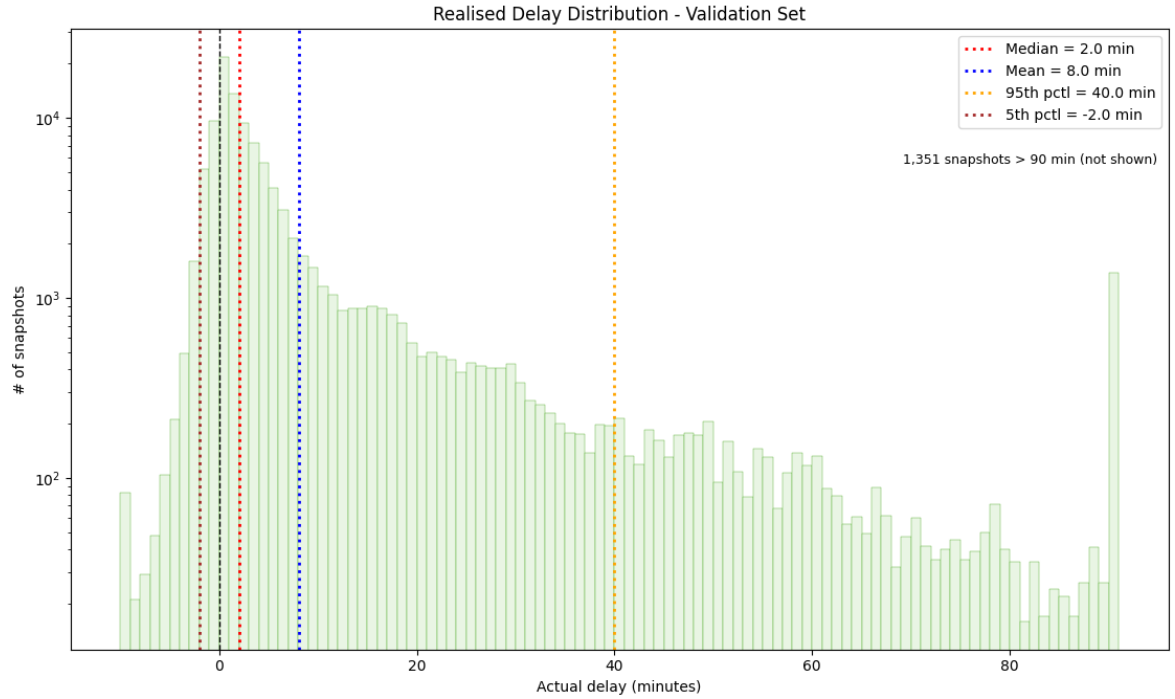


Figure 4.2: Distribution of realised delays in the validation set.

The figure above shows actual delays from -10 minutes through 90 minutes, and the number of snapshots that hold these delays. For readability, the x-axis is clipped to `[-10, 90]` minutes. Snapshots with delays exceeding 90 minutes and with delays below -10 minutes are aggregated into the leftmost and rightmost bars respectively. The mean delay is 8 minutes, and median 2 minutes. The 90% distribution interval is between the lower bound -2 minutes and the upper bound 40 minutes. This skewed distribution is why aggregate metrics like RMSE alone are insufficient and why stratification by delay bins is necessary for transparency and understanding.

4.1.4 Forecast horizon distribution

The distribution of forecast horizon (ATA – snapshot time) is plotted below in Figure 4.3.

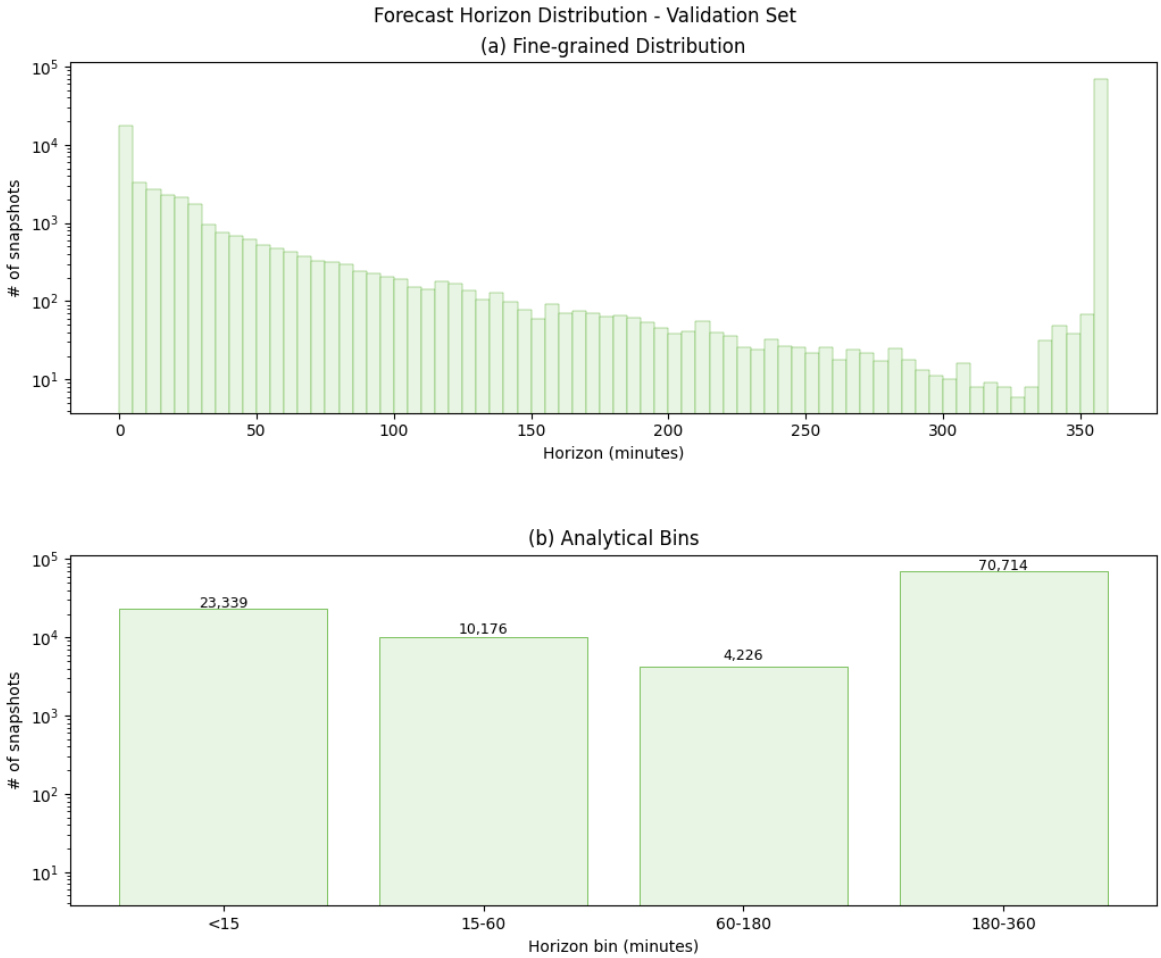


Figure 4.3: Distribution of forecast horizon across the validation set. (a) – Fine-grained histogram with 5-minute bins. (b) – Histogram with analytical bins used throughout this chapter.

The figure above shows the distribution of forecast horizon from 0 minutes up to 6 hours, and the number of snapshots that hold these horizons. Subplot (a) shows a fine-grained histogram with 5-minute bins, where the distribution has a smooth decay from horizon zero down to about 280 minutes, a sparsely populated section around and after 300 minutes, and a sharp concentration at 355-360 minutes. The population spike around the 6 hour horizon reflects the snapshot collector's cadence of emitting a first observation approximately six hours before each event's ATA. Subplot (b) shows the same data aggregated into the analytical bins mentioned in Section 3.5.5. `70,714` snapshots fall into the 180-360 minute bin, roughly 65%. This imbalance means aggregate metrics are dominated by this bin, motivating stratified analysis.

4.1.5 Trafikverket baseline behaviour

The distribution of Trafikverket's predicted delays is plotted below in Figure 4.4.

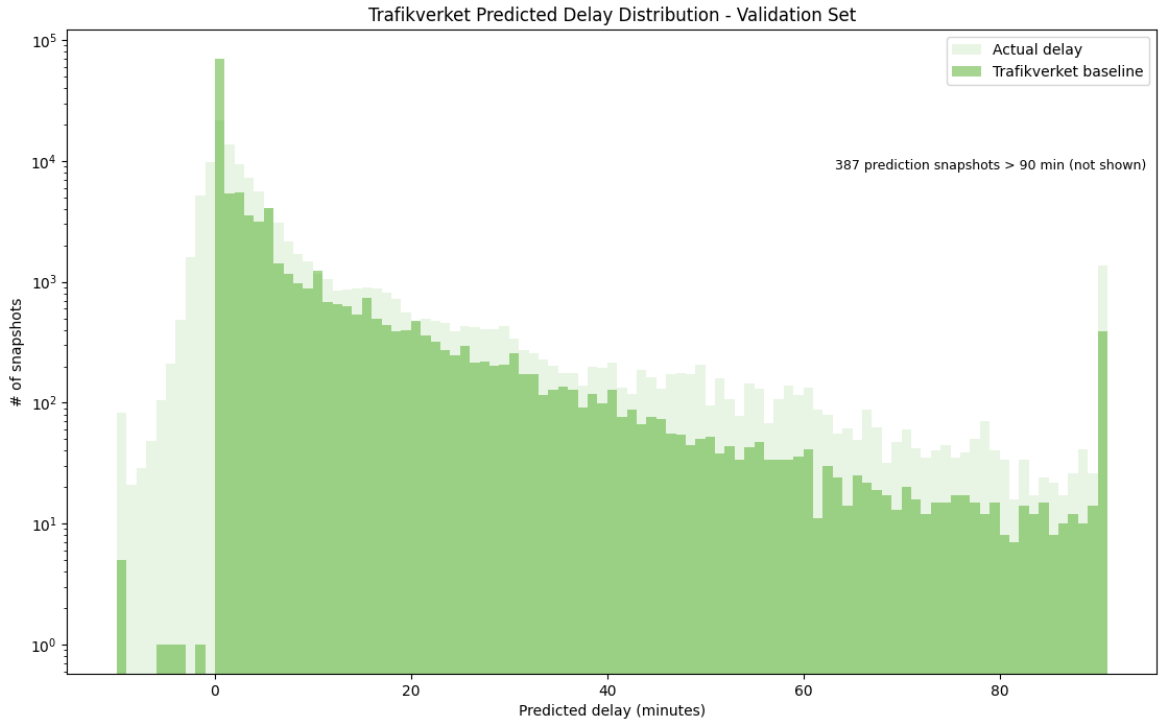


Figure 4.4: Distribution of predicted delay from Trafikverket across the validation set.

The figure above shows the distribution of Trafikverket's predicted delays from -10 minutes through 90 minutes. There are 387 snapshots that has a predicted delay from Trafikverket of over 90 minutes. Trafikverket overwhelmingly predicts 0 minutes delay, under-predicts delays under 0 minutes (where trains arrive before the ATA), and Trafikverket's tail above 40 minutes is also notably thinner than the realised tail. The mean predicted delay is 4 minutes, and the median 0 minutes.

4.1.6 Summary

Below, in Table 4.1, is a summary of descriptive statistics from the validation set.

Table 4.1: Summary of descriptive statistics on validation set.

Statistic	**Value**
Total snapshots	108,455
Unique activities	69,852
Median snapshots / activity	1
Mean snapshots / activity	1.55
Median actual delay	2 min
Mean actual delay	8 min
95th percentile actual delay	40 min
% horizon $<$ 15 min	21.5%
% horizon 180-360 min	65.2%
Median Trafikverket predicted delay	0 min
Mean Trafikverket predicted delay	4 min

4.2 Model Performance

This section provides the results of the overall model performance for all three models compared against the baseline.

4.2.1 Aggregate Point-Prediction Performance

Below, in Table 4.2 are the aggregate error metric results for the point-prediction models.

Table 4.2: Aggregate point-prediction error metrics.

Model	$RMSE$ (min)	MAE (min)	R^2	Skill
LightGBM	**10.67**	**4.47**	**0.742**	**+0.241**
Random Forest	11.04	4.64	0.724	+0.215
Ridge	12.59	5.05	0.640	+0.104
Trafikverket ETA (baseline)	14.05	4.90	0.552	—

All models show positive skill against the baseline, with LightGBM as the model with the highest skill, followed by Random Forest and then Ridge. LightGBM had a 10.67 minute $RMSE$, representing a 24% reduction over the operational ETA. Random Forest had a 11.04 minute $RMSE$, a 21% reduction that's close to LightGBM but with a meaningful gap to Ridge. Ridge had a 12.59 minute $RMSE$, a 10% reduction

that narrowly beats the baseline, confirming even linear correction adds value. MAE values generally follow the $RMSE$ trend, other than Ridge whose MAE was 5.05 – a worse score than the baseline's (4.90) despite the error reduction found in its $RMSE$. This is a property of skill gain over greater horizons that is further explained in Section 4.3.2. R^2 follows the pattern set by $RMSE$ as well, with LightGBM explaining about 74% of target variance, Random Forest about 72%, and Ridge 64%, compared to the R^2 of the baseline which is around 55%.

The skill rankings from Table 4.2 matches model complexity, where linear regression is beat by bagging methods used by Random Forest, that in turn is beat by the gradient boosting in LightGBM.

4.2.2 Prediction Interval Performance

Table 4.3 below shows prediction interval diagnostics.

Table 4.3: Prediction Interval Diagnostics (90% nominal, [5%, 95%] quantile).

Model	Coverage	Mean width (min)	Median width (min)	Point outside interval
LightGBM	90.7%	17.35	10.28	0.57%
Random Forest	27.6%	5.84	1.74	0.00%
Ridge	80.3%	13.55	8.51	21.85%

The main finding of prediction interval performance shows three qualitatively different failure modes. LightGBM has a coverage of 90.7%, near the nominal 90% target. Its interval widths reflect the true uncertainty as the only model with usable intervals as-is. Its mean width is 17.35 minutes and median 10.28 minutes, which means widths are right-skewed and tight when the model is confident, wide when unsure. The percentage of point predictions being outside interval bounds are 0.57%.

Random Forest has a coverage of 27.6%, about a third of the 90% target. This is a severe under-coverage. While mean and median widths are tight (5.84 minutes and 1.74 minutes), and point-outside-interval percentage is 0%, these values are dishonest since RTA most often lands outside these bounds.

Ridge has a coverage of 80.3%, which sits slightly below the nominal target. Mean and median widths are 13.55 minutes and 8.51 minutes, respectively. Point predictions fall out of interval bounds 21.85% of the time.

4.2.3 Summary

In summary, all three models improve on the operation ETA on RMSE-skill, but only LightGBM and Random Forest improve on MAE. Skill rankings follow model complexity. Point prediction quality and interval quality do not co-vary – Ridge has tighter widths than LightGBM but broken intervals, and Random Forest has the tightest widths but a low coverage. LightGBM is the only models that delivers both.

4.3 Error Analysis

In Section 4.2 aggregate metrics and prediction intervals are displayed. This section decomposes the point prediction errors in order to characterise where and how models err.

4.3.1 Residual Distribution

Figure 4.5 below shows the error distribution across snapshots of all models.

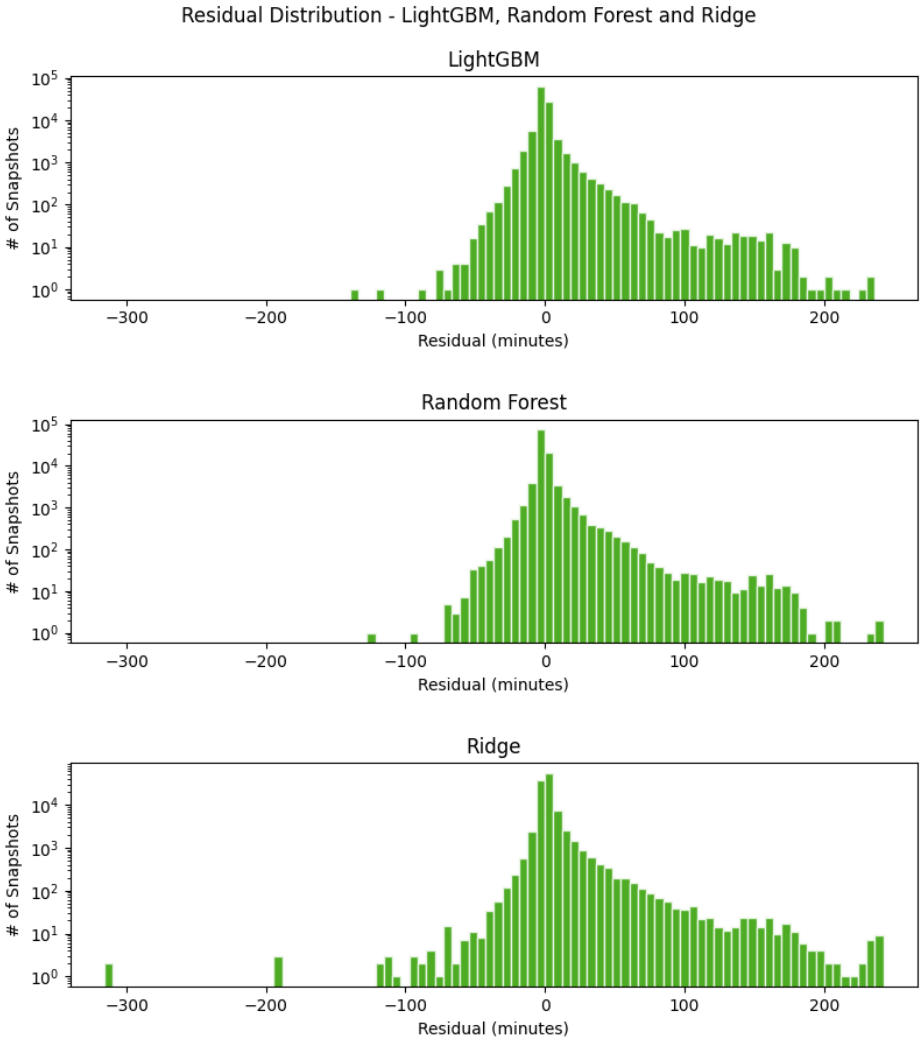


Figure 4.5: Distribution of model residuals.

All three models concentrate mass near zero minutes and a right under-prediction tail, but with slightly different shapes. LightGBM has the narrowest, most symmetrical shape, consistent with its $RMSE$ performance. Random Forest shows a similar shape with slightly heavier shoulders. Finally, Ridge has a noticeably more visible asymmetry between near-zero mass and under-prediction tail, as well as larger error near the 200-minute under-prediction mark and a longer left tail as well.

4.3.2 Error by Forecast Horizon

Tables 4.4, 4.5 and 4.6 below show model point prediction error stratified by forecast horizon bins introduced in Section 3.5.5.

Table 4.4: Error by forecast horizon - LightGBM.

Horizon (min)	Share	$RMSE$	Baseline $RMSE$	Skill	MAE	Bias
0-15	21.5%	6.85	8.37	+0.1816	3.04	+0.12
15-60	9.4%	13.12	16.19	+0.1899	6.58	-0.14
60-180	3.9%	21.97	33.07	+0.3357	14.40	+0.71
180-360	65.2%	10.27	13.29	+0.2277	4.05	-0.13

Table 4.5: Error by forecast horizon - Random Forest.

Horizon (min)	Share	$RMSE$	Baseline $RMSE$	Skill	MAE	Bias
0-15	21.5%	6.91	8.37	+0.1744	3.19	-0.50
15-60	9.4%	13.33	16.19	+0.1765	6.71	-0.10
60-180	3.9%	22.61	33.07	+0.3164	14.95	+1.91
180-360	65.2%	10.72	13.29	+0.1935	4.20	+0.05

Table 4.6: Error by forecast horizon - Ridge.

Horizon (min)	Share	$RMSE$	Baseline $RMSE$	Skill	MAE	Bias
0-15	21.5%	8.02	8.37	+0.0422	4.25	+2.05
15-60	9.4%	14.35	16.19	+0.1140	6.81	+3.36
60-180	3.9%	27.18	33.07	+0.1781	17.55	+11.82
180-360	65.2%	12.17	13.29	+0.0842	4.31	+2.53

Skill is lowest in the 0-15 minute bin across all models. Since the baseline has an $RMSE$ of 8.37 minutes in this bin, there is little headroom for the models to improve. Skill peaks in the 60-180 minute bin for all models. The baseline $RMSE$ degrades to 33.07 minutes, but model $RMSE$ stays in the 20's, and is where the models add the most value. At the dominant bin, 180-360 minutes (65.2% of snapshots), skill is still positive.

Bias seems to separate models more clearly than $RMSE$ alone. For LightGBM, bias stays within ± 1 minute in every bin. Random Forest bias is also well-centred at ± 0.5 minutes except the 60-180 minute bin, where it under-predicts by 1.91 minutes. Ridge under-predicts in every bin. Bias grows roughly monotonically with horizon, peaking at +11.82 minutes in the 60-180 minute bin.

Table 4.6 above shows why Ridge had a better aggregate $RMSE$ than the baseline but a worse MAE (see Table 4.2). $RMSE$ rewards corrections most where baseline errors are largest, the long horizon bins where Ridge's gains concentrate. MAE weighs every snapshot equally, so the short-horizon bins drag Ridge's MAE up.

4.3.3 Error by Delay Magnitude

Tables 4.7, 4.8 and 4.9 below show model point prediction error stratified by delay magnitude bins introduced in Section 3.5.5.

Table 4.7: Error by delay magnitude - LightGBM.

Bin	Share	$RMSE$	Baseline $RMSE$	Skill	MAE	Bias
`Early`	16.1%	5.44	2.56	-1.1250	3.75	-3.75
`On-time`	53.2%	3.58	1.81	-0.9762	2.06	-1.57
`Minor delay`	16.0%	5.56	6.12	+0.0907	3.74	+0.32
`Major delay`	12.2%	13.93	18.82	+0.2601	9.96	+3.90
`Severe delay`	2.6%	53.51	75.26	+0.2890	37.43	+33.61

Table 4.8: Error by delay magnitude - Random Forest.

Bin	Share	$RMSE$	Baseline $RMSE$	Skill	MAE	Bias
`Early`	16.1%	5.13	2.56	-1.0022	4.15	-4.15
`On-time`	53.2%	3.07	1.81	-0.6915	1.99	-1.75
`Minor delay`	16.0%	5.33	6.12	+0.1294	3.62	+0.77
`Major delay`	12.2%	14.83	18.82	+0.2121	10.66	+4.59
`Severe delay`	2.6%	56.23	75.26	+0.2529	40.17	+35.12

Table 4.9: Error by delay magnitude - Ridge.

Bin	Share	$RMSE$	Baseline $RMSE$	Skill	MAE	Bias
`Early`	16.1%	3.35	2.56	-0.3078	2.24	-1.50
`On-time`	53.2%	2.99	1.81	-0.6511	2.20	+0.65
`Minor delay`	16.0%	6.71	6.12	-0.0972	4.70	+3.65
`Major delay`	12.2%	16.72	18.82	+0.1118	12.35	+9.15
`Severe delay`	2.6%	65.64	75.26	+0.1278	48.97	+41.19

All three models show negative skill in the `Early` and `On-time` bins, which means the operational ETA has a lower $RMSE$ in these bins. Baseline $RMSE$ in the `On-time` bin is 1.81 minutes. When no disruption is present, the ETA from Trafikverket is nearly perfect, and none of the models can improve on it without introducing noise elsewhere. The baseline degrades sharply as delay grows. $RMSE$ goes from 1.81 (`On-time`) to 6.12 (`Minor delay`), 18.82 (`Major delay`), and all the way to 75.26 (`Severe delay`). This is where models recover their aggregate skill. Random Forest and LightGBM reach +0.21 to +0.29 in the `Major delay` and `Severe delay` bins.

All models seem to also systematically under-predict severe delays. The biases are +33.61 minutes for LightGBM, +35.12 for Random Forest and +41.19 minutes for Ridge. They expect, on average, `Severe delay` trains to be about 34-41 minutes less late than they turn out to be. This is an effect of $RMSE$ training paying a heavy penalty for over-shooting the dominant on-time mass, so the optimisation compromises by shrinking severe predictions back towards the centre. Skill stays positive in the `Severe delay` bin, however, due to the baseline under-predicting even more heavily (75.26 minutes).

Along with under-predicting severe delays, all models seem to systematically over-predict early arrivals. For the `Early` bin, LightGBM has a bias of -3.75 minutes, Random Forest -4.15 minutes and Ridge -1.50 minutes. This is a mirror image of the `Severe delay` pattern, where the same effect that under-shoots severe delays also drags early-arrival predictions upward towards zero. The baseline's advantage with an $RMSE$ of 2.56 minutes reflects that the ETA from Trafikverket captures especially 0 minute delays well, and the models cannot compete.

4.3.4 Summary

All three models implement the same underlying behaviour, improving the operational ETA at longer horizons and on larger delays. Differences between models are in fidelity. LightGBM has the lowest horizon-conditional bias, highest skill in the delay tail and smallest under-prediction of severe delays. Random Forest is a close second with a similar horizon profile and a slightly larger under-prediction in the delay tail. Ridge shares the same qualitative pattern but with substantially larger under-prediction biases at long horizons and large delays.

The operational ETA wins on the dominant on-time and early-arrival mass. None of the models improve the near-perfect baseline without introducing noise elsewhere. All three models systematically under-predict severe delays by 34-41 minutes on average, which is a factor of squared-error training on a right-skewed target. Skill stays positive

in this bin only because the operational ETA under-predicts even more heavily.

4.4 Feature Importance

The following section covers the results of the feature importance methods used for each model, in order to answer RQ2 (see Section 1.1).

4.4.1 Permutation Importance

Permutation importance was used to calculate global importance across all features. For each model, 5,000 test rows were sampled and used to determine which features had the biggest impact on $RMSE$. In Tables 4.10, 4.11 and 4.12 below, permutation importance is shown for the top five features of each model. The limit of five features was chosen as a presentation cut-off, and the full ranked list for each model is available in Tables A.3, A.4 and A.5.

Table 4.10: Permutation Importance (5000 samples) - LightGBM.

Feature	$\Delta RMSE$ (mean)	Std
`eta_delay_minutes`	8.4319	0.0973
`horizon_minutes`	4.0471	0.1511
`eta_delay_lag1`	1.7801	0.0421
`eta_delay_delta`	1.6200	0.0489
`location_signature`	1.1001	0.0259

Table 4.11: Permutation Importance (5000 samples) - Random Forest.

Feature	$\Delta RMSE$ (mean)	Std
`eta_delay_minutes`	7.2460	0.1061
`horizon_minutes`	3.0161	0.1067
`eta_delay_lag1`	1.9511	0.0533
`eta_delay_delta`	1.5953	0.0482
`eta_delay_rolling_mean_3`	1.0719	0.0327

Table 4.12: Permutation Importance (5000 samples) - Ridge.

Feature	$\Delta RMSE$ (mean)	Std
`eta_delay_lag1`	6.2322	0.0929
`eta_delay_minutes`	5.5869	0.0869
`eta_delay_rolling_mean_3`	0.5717	0.0440
`temp_avg_daily`	0.3164	0.0199
`horizon_minutes`	0.2022	0.0176

There is agreement across models, where `eta_delay_minutes` is a top two feature for all models. This is the operational ETA baseline delay. For LightGBM and Random Forest, this feature has the largest impact on $RMSE$ after shuffling, changing it by an average of 7-8 minutes. For Ridge, it shows a slightly lower importance, impacting $RMSE$ by about 6 minutes. The ETA history features (`eta_delay_lag1`, `eta_delay_rolling_mean_3` and `eta_delay_delta`) also show up in the top five features, showing that historic ETA evolution carries meaningful predictive signal. The current horizon, `horizon_minutes`, is the number two most important feature for LightGBM and Random Forest, and takes a fifth place for Ridge. The train station signature, `location_signature`, shows up as the fifth most important feature for LightGBM, but does not show up for Random Forest and Ridge. This is a common limitation of one-hot encoding, where categorical features get split into their available values and therefore show importance per value instead of the category as a whole, making `location_signature` distribute differently. While `temp_avg_daily` appears among Ridge's feature list, other weather features, operator identity and time-of-day variables do not appear in the top five for any model. Their permutation importance values are below 0.1 $\Delta RMSE$ across all three models.

4.4.2 SHAP Values

SHAP values were computed on a 2,000-row sample of the test set for each model. Unlike permutation importance, SHAP attributes each prediction to its features with both magnitude and direction, supporting local explanations as per-prediction breakdowns. For each model, local explanations for the highest, lowest, and median predictions in the sampled set is reported below in Tables 4.13, 4.14 and 4.15.

Table 4.13: Local SHAP explanations – LightGBM (base value = 9.65 min).

Feature	SHAP contrib (min)	Feature value
Highest prediction: 231.46 min		
`eta_delay_minutes`	+136.302	194.0
`eta_delay_lag1`	+51.130	188.0
`eta_delay_lag2`	+15.424	178.0
`eta_delay_rolling_mean_3`	+11.878	175.0
`location_signature`	+4.500	Cst
`eta_delay_delta`	+3.081	6.0
`day_of_week`	-2.714	2
`minutes_since_last_snapshot`	+1.642	225.0
Lowest prediction: -0.61 min		
`eta_delay_minutes`	-5.320	0.0
`horizon_minutes`	+3.289	358.99
`location_signature`	-2.884	Gal
`eta_delay_delta`	-1.830	0.0
`temp_avg_daily`	-1.280	4.6
`eta_delay_lag1`	-1.265	0.0
`wind_gust`	+0.679	11.7
`is_arrival`	-0.679	1
Median prediction: 2.49 min		
`eta_delay_minutes`	-3.368	2.0
`location_signature`	-1.353	Kac
`eta_delay_lag1`	-0.936	0.0
`is_arrival`	-0.584	1
`horizon_minutes`	-0.388	24.98
`temp_avg_daily`	-0.355	-0.3
`eta_delay_rolling_mean_3`	-0.322	0.0
`eta_delay_delta`	+0.252	2.0

Table 4.14: Local SHAP explanations – Random Forest (base value = 4.15 min).

Feature	SHAP contrib (min)	Feature value
Highest prediction: 229.68 min		
eta_delay_minutes	+99.025	194.0
eta_delay_lag1	+53.416	188.0
eta_delay_rolling_mean_3	+31.936	175.0
eta_delay_lag2	+23.443	178.0
eta_delay_delta	+8.172	6.0
n_snapshots_seen	+3.680	5.0
minutes_since_last_snapshot	+2.627	225.0
horizon_minutes	+1.618	47.99
Lowest prediction: 1.67 min		
horizon_minutes	+2.389	356.99
eta_delay_minutes	-1.504	0.0
eta_delay_lag2	-0.789	0.0
eta_delay_rolling_mean_3	-0.759	0.0
eta_delay_lag1	-0.631	0.0
temp_avg_daily	-0.408	4.0
is_arrival	-0.408	1.0
dew_point	-0.377	-7.7
Median prediction: 2.75 min		
horizon_minutes	+2.593	355.99
eta_delay_minutes	-1.487	0.0
hour	+1.108	17.0
eta_delay_lag2	-0.874	0.0
eta_delay_rolling_mean_3	-0.835	0.0
eta_delay_lag1	-0.692	0.0
temp_avg_daily	-0.483	5.8
is_arrival	-0.347	1.0

Table 4.15: Local SHAP explanations – Ridge (base value = 1.39 min).

Feature	SHAP contrib (min)	Feature value (scaled)
Highest prediction: 255.08 min		
`eta_delay_lag1`	+167.166	14.13
`eta_delay_minutes`	+149.531	12.80
`eta_delay_rolling_mean_3`	-83.241	15.34
`eta_delay_lag2`	+12.223	14.67
`eta_delay_delta`	+3.296	0.82
`location_signature_Cst`	+2.627	6.58
`n_snapshots_seen`	+1.763	2.05
`minutes_since_last_snapshot`	+1.715	1.28
Lowest prediction: -5.26 min		
`temp_avg_daily`	-4.157	2.65
`minutes_since_last_snapshot`	+2.708	2.31
`location_signature_Vg`	-2.515	11.01
`eta_delay_lag1`	-1.263	-0.26
`is_arrival`	-0.898	0.94
`eta_delay_rolling_mean_3`	+0.664	-0.23
`eta_delay_delta`	+0.493	-0.10
`hour`	-0.460	-1.08
Median prediction: 0.98 min		
`temp_avg_daily`	-3.537	2.24
`minutes_since_last_snapshot`	+2.747	2.35
`eta_delay_lag1`	-1.263	-0.26
`eta_delay_rolling_mean_3`	+0.664	-0.23
`is_arrival`	+0.599	-1.06
`eta_delay_delta`	+0.493	-0.10
`n_snapshots_seen`	+0.414	0.09
`eta_delay_minutes`	-0.412	-0.27

For the highest prediction (around 230-255 minutes of delay across all three models), the same event in the test set is being explained, a train with a published ETA of 194 minutes late, where all three models predict a further escalation. The contributions are dominated by the operational ETA and its history: `eta_delay_minutes`, `eta_delay_lag1`, and `eta_delay_rolling_mean_3` together account for nearly all of the prediction's distance from the base value (what the model would have predicted

based on training alone and no features) in every model.

For the lowest and median predictions, the dominant features differ across models. In LightGBM and Random Forest, low predictions are driven primarily by a low `eta_delay_minutes` value (0.0 in both cases) pulling predictions below the base value, with `horizon_minutes` acting as a counterweight at long horizons. In Ridge, the same low-prediction event is dominated by `temp_avg_daily`, which could reflect a collinearity, where the linear model spreads the operational ETA signal across several correlated features, as mentioned in Section 2.3.5.

4.4.3 Summary

The two feature importance methods broadly agree across all three models on which features drive predictive performance. The operational ETA (`eta_delay_minutes`) and the forecast horizon (`horizon_minutes`) are top-three features in every permutation importance ranking, with the ETA-history group (`eta_delay_lag1`, `eta_delay_delta`, `eta_delay_rolling_mean_3`) consistently appearing in the top five. The local SHAP explanations support the same conclusion at the per-prediction level. For the highest-prediction event in the test set, these features alone account for the bulk of the deviation from each model's base value.

What is also informative is what does not appear. Most weather features, day of week, and month all rank mostly below 0.1 $\Delta RMSE$ in permutation importance across all three models. Short-horizon delay forecasting in this dataset is dominated by the current operational state of the train rather than by exogenous factors.

Chapter 5

Discussion

The following chapter discusses the results gained, answering all research questions (see Section 1.1), reflecting on practical implications, discussing limitations and ideating potential future work.

5.1 Interpretation of Results

This section covers the results of the thesis as they relate to the research questions from Section 1.1, what improved, what did not, and what actually mattered.

5.1.1 Predictive Accuracy

All three models beat Trafikverket's operational ETA on $RMSE$. The aggregated skill scores are +0.10 for Ridge, +0.21 for Random Forest and +0.24 for LightGBM (see Table 4.2). This means the skill ranking matches model complexity, where Ridge is the simplest, followed by Random Forest as a middle ground and LightGBM as the most complex. This finding means that for this task, where many features interact dynamically, the more complex boosting method yields the most accurate predictions, beating both linear regression and bagging, aligning with previous findings of Section 2.1.4 [23].

At first glance, the R^2 scores look impressive for each model but are partly inherited from the ETA, because it was used as a feature. LightGBM had an R^2 value of 0.742, but compared to the baseline R^2 (0.552) it is only a 0.190 point increase (see Table 4.2), which itself is a measure of ΔR^2 – a metric difficult to decompose cleanly and interpret. Therefore, skill is the more important metric for evaluating the models compared to the baseline. It measures the correction the model adds on top of the

published forecast. The LightGBM skill score of 0.24 indicates a 24% reduction in RMSE relative to the operational baseline, which is the more interpretable measure for evaluating the correction the model adds.

Skill is concentrated in the delay tail. In the `On-time` bin (see Section 4.3.3), which is 53% of validation rows, all three models underperform the ETA baseline. Skill turns positive only at `Minor delay` (except for Ridge that still has a negative skill) and grows monotonically through `Severe delay`. Bias turns negative in the `Early` and `On-time` bins, which could be due to the baseline being very strong for on-time trains. Instead of agreeing with the baseline, the models may want to reduce error further, but instead introduce noise (an overfit caveat as described through Section 2.3.2). When delay severity increases, the baseline fails, which is where the models dominate skill. However, bias is still large in the `Severe delay` bin for every model. Since the bin is only 2.6% of the validation rows, the models are not very good at predicting severe delays, which is further addressed in Section 5.3.4. The baseline is, however, significantly worse, which is why skill is the highest here. Overall, LightGBM performs the best in terms of skill in the `Major delay` and `Severe delay` bins, while Ridge actually outperforms the others in the `Early` and `On-time` bins. Ridge's predictions in these bins deviate less from the published ETA than LightGBM's and Random Forest's do, consistent with limited capacity to overfit small residuals (again described through Section 2.3.2). While $RMSE$ and MAE for all models are at their lowest score in the `On-time` bin, the baseline alone outperforms all of them, meaning there is little practical value for the models here.

Skill is also concentrated at larger horizons, and mainly grows with it increasingly. All three models outperform the baseline at every horizon (see Section 4.3.2), and the value comes from the baseline degrading faster at long horizons than the models. This aligns with previous studies of estimation accuracy over longer horizons (Spanninger, 2023) as discussed in Section 2.4.3 [37]. A majority of validation rows (65%) fall in the `180-360` minute bin, but skill is not highest here. Most of these are one-row events (since most trains are on-time), where no further ETA updates became available after the initial fetch. The dominance of single-row events in this bin reflects a selection effect. Events with stable, well-behaved ETAs by 3-6 hours before arrival are over-represented because they generated only a single snapshot. The reported skill in this bin should therefore be read as conditional on this subset rather than as fully representative of long-horizon performance. This means ETA history was unavailable as features, lowering the skill across all models (LightGBM at +0.2277, Random Forest at +0.1935 and Ridge at +0.0842), and is further discussed in Section 5.3.3. The models all perform their best at the `60-180` minute bin, where skill grows to +0.3357 for LightGBM, +0.3164 for

Random Forest and +0.1781 for Ridge. At this bin, there are enough event updates for the models to learn from ETA history, but the baseline itself still struggles with forecasting at a large horizon. These factors both make skill increase. Another factor could be the degradation of the baseline itself. The baseline reaches its largest $RMSE$ at this horizon, a score of 33.07. While the models also have their largest $RMSE$ here (21.97 for LightGBM, 22.61 for Random Forest and 27.18 for Ridge), the margin between the two is where skill value comes from. At the smaller horizons (`0-15` and `15-60` minutes), skill is relatively stable at around +0.18 and +0.17 for LightGBM and Random Forest, respectively. Ridge instead has a larger margin, with around +0.04 at the `0-15` bin and around 0.11 at the `15-60` minute bin. This could be a factor of the baseline degradation, which makes Ridge look like it gets three times as good from one horizon to the other. Overall, LightGBM outperforms the other models at every horizon, followed by Random Forest and then Ridge. This means skill ranking matches model complexity not only aggregated, but stratified by forecast horizon, aligning with previous findings of Section 2.1.4 [23]. The preservation of the complexity-skill ordering across both delay severity and forecast horizon stratifications suggests the additional capacity of LightGBM is being used productively rather than overfits. When it comes to $RMSE$ and MAE, all models have their best scores at the `0-15` minute bin. This is where model accuracy is highest without comparison to the baseline, but again holds little value due to the baseline also being close in $RMSE$, and also leaves little room for meaningful intervention at this bin as explained in Section 2.1.4 [37].

In summary, LightGBM generally outperforms the other models in aggregated accuracy, accuracy stratified by delay severity and accuracy stratified by forecast horizon. The aggregated $RMSE$ for the model is 10.67 minutes, MAE is 4.47 minutes and R^2 is 0.742. This is a skill increase of +0.24 over the operational ETA, and means a large share of the model's predictive power is inherited from the operational ETA used as a feature while still adding a meaningful correction on top of it. Other than Ridge being the top performer in the `Early` and `On-time` delay bins, LightGBM's complexity proves beneficial in both stratification tables. All models show their highest skill in the `60-180` minute horizon bin and the `Severe delay` bin, meaning these cases are where the models hold the most value over the baseline. In terms of overall accuracy, all models are their most accurate at the `0-15` minute horizon bin and the `On-time` delay bin. This is consistent with accuracy increasing as horizon shrinks, and on-time trains dominating the dataset, making them them more common for the models to predict. However, since the baseline actually outperforms all models in the `On-time` delay bin, the models hold no value here. Similarly, the `0-15` minute horizon bin is where the baseline is strongest, meaning there is not much room for improvement for the models. The value the models add is therefore concentrated in the operationally important

cases – long horizons where the baseline degrades, and significant delays where the baseline is unreliable. These are precisely the conditions under which a passenger or operator may need an improved estimate the most, as described in Section 2.1.4.

5.1.2 Features and Modelling

SHAP local explanations and permutation importance as global explanations mostly agree across all three models on the main features ranked by importance (see Sections 4.4.1 and 4.4.2). The generally most important features are the operational ETA itself, the forecast horizon, and the ETA history group (lag, change in delay and rolling mean). When first constructing the dataset, snapshots were grouped per event, with each event represented by a single row containing only its latest available snapshot. The dataset was later redesigned to be snapshot-keyed, in order to expose ETA evolution as a signal, and the fact that these features rank highly across both importance methods is empirical validation of this design choice. The previous static design did not expose ETA history features at all, since each event collapsed to a single row containing only the most recent snapshot. The snapshot-keyed design makes these features available by construction, and their high importance ranking confirms that the redesign successfully exposed the temporal signal that motivated it.

Most weather features rarely contribute much to predictions, ranking near-zero in all three models. This is consistent with findings of short horizon delay forecasting (Spanninger, 2023), where the current operational state dominates external features [37]. Furthermore, this may also reflect relatively static weather conditions with little variation during the collection period, which is further discussed in Section 5.3.1.

A feature that appears as a top one for only LightGBM is `location_signature`. The reason this feature is not as present for the other models is their one-hot encoding where categorical features, before training, get split into multiple features by their available values, as explained in Section 4.4.1. This makes the categorical feature as a whole have a signal less strong. Since Random Forest and Ridge does not have native handling of categorials, they must be encoded this way, which makes the feature appear to contribute less. This caveat is further discussed as a limitation in Section 5.3.6.

The issue of collinearity introduced in Section 2.3.5 is now empirically grounded by the permutation importance results. Ridge's largest permutation importance is on `eta_delay_lag1` (6.23 $\Delta RMSE$) and not `eta_delay_minutes`. This is precisely the type of attribution shuffling between correlated features that collinearity entails. The two features carry nearly identical information about the operational ETA at successive snapshots, which makes them interchangeable.

The feature holding the delay for the snapshot at the previous station in the journey, `prev_dep_delay`, ranks near the bottom for all models and both importance methods, but this was not always the case during implementation. When evaluating early model results, a leakage was discovered from the feature in how the delay of the upstream departure was computed. It was derived from the RTA of the upstream event, the ground truth observed delay, computed after that departure had happened. The value was then broadcast identically across every snapshot of the downstream event. The intent of this was operationally sensible, since knowing how delayed the upstream departure was should help estimate the current arrival. This, however, meant that if a snapshot of the arrival event at station A was fetched at for example 13:00, and the train's upstream departure from station B was not scheduled until 13:30, the feature would still contain the upstream departure's eventual realised delay which is information that did not yet exist at 13:00. The model was therefore trained on snapshots where one of its input features told it the answer to a question that would have been unknown at the snapshot's fetch timestamp. This feature was able to see the future, and therefore ranked as the number one feature across all metrics and models, even returning an inflated R^2 of 0.94 and skill of +0.64 for LightGBM – values that should be understood as the diagnostic that prompted investigation, not as a result. After discovery, the corrected `prev_dep_delay` was rebuilt to use only information available at each snapshot's fetch timestamp, making the feature now represent a real time estimate available at prediction time. This was the more honest representation of the feature, but reduced its importance ranking and also brought the overall metric scores down to the values reported throughout this thesis. Standard temporal train/test splits did not catch this leakage, as the issue lived within each row's feature vector rather than between train and test partitions. In multi-step time-series pipelines where one event's outcome becomes another event's input, leakage can hide inside derivations that appear operationally reasonable, and aggregate metrics that look unusually strong should be treated as a diagnostic signal rather than a result.

The main modelling choice affecting results was the previously mentioned snapshot-keyed dataset design. The decision to restructure the dataset from event-collapsed to snapshot-keyed allowed models to learn from ETA history of previous snapshots for the same event, which turned out to be among the top features, a feature engineering approach with downstream effects on every model. Another modelling approach that was vital for not inflating metrics and portraying honest results was the temporal cross-validation strategy. Using time-series CV rather than randomized k-fold CV (as described in Section 3.5.2 and reasoned in Section 2.3.4) allowed the data to be temporally ordered. In a time-series context, this is the difference between honest results and inflation due to data leakage, as the paragraph above also shows. Additionally,

the `prev_dep_delay` leakage discovery was not only a feature level issue but a finding teachable for modelling approaches in future works.

Hyperparameter optimisation was another approach that affected runtime and robustness. In Section 2.3.4, three variations are discussed. For LightGBM and Random Forest, Random Search was chosen as the method, which cut the complexity cost of Bayesian Optimisation while still being less computationally heavy than Grid Search. This was especially important for the tree ensembles since they have many hyperparameters that can be tweaked. Ridge only has one hyperparameter, which allowed it to instead use Grid Search. The total number of iterations during hyperparameter tuning was 150 for LightGBM and Random Forest, and 35 for Ridge. This meant to ensure confident hyperparameter values before full scale training, but choosing the proper thresholds has an impact on results. This is further discussed in Section 5.3.7.

As discussed, there is an encoding asymmetry for categorical features. This was a modelling choice with measurable consequences as LightGBM is the only model with native categorical handling, that the other models do not have access to (as further discussed in Section 5.3.6).

5.1.3 Prediction Interval Implications

All three models were configured to produce a nominal 90% prediction interval [5%, 95%], but the empirical behaviour of these intervals diverges substantially (see Table 4.3). They differ to the point where the three models exhibit three qualitatively distinct failure modes rather than three points on a single quality axis. LightGBM used quantile regression, and has an empirical coverage of 90.7% at 90% nominal, median width of 10.28 minutes, and crossover rate of 0.57%, making it the only model that produces intervals fit for direct use. The two other models each fail in a different and methodologically informative way. Random Forest used tree-prediction-percentiles, and has an empirical coverage of 27.6%, median width of 1.74 minutes and crossover rate of 0.00%. The empirical coverage and mean width makes the model severely under-cover with very narrow intervals. Ridge, using independent quantile fit, has a different issue. It has a coverage of 80.3% and median width of 8.51 minutes, but its crossover rate is 21.85%. This means that about a fifth of point predictions fall outside the interval, failing the crossover check and rendering these intervals unusable. LightGBM is therefore the only model the interval bounds can be applied to practically.

Quantile regression (LightGBM) is trained as a loss function level objective. The model directly optimises for the desired quantile of the conditional distribution, so the resulting bounds reflect the model's actual estimate of uncertainty around each prediction. This

is what makes the coverage nominal. Tree-prediction-percentile (Random Forest), on the other hand, takes the fifth and 95th percentiles of the predictions across the trees in the ensemble for each test points. This measures model variance across trees, not the irreducible noise in realised delays. In a well trained Random Forest, trees agree closely on individual predictions, so the resulting interval is narrow even when the actual realised delay could be anywhere. The intervals appear confident but are not justified. In independent quantile fit (Ridge), the lower bound, point estimate and upper bound are fitted as three independent quantile-regression models. There is therefore here no constraint enforcing that there is no crossover for any given prediction. Combined with collinearity between features (as explained in Section 2.3.5, the three estimators can disagree on the rank ordering of test points, producing 21.85% of cases where the point prediction falls outside the model's own interval, a self-inconsistent outcome that is geometrically impossible in a well-constructed interval.

Thus, the uncertainty estimates are not a single methodological choice. The same nominal goal yields radically different empirical behaviour depending on how the interval is constructed. This matters because uncalibrated intervals are not just less useful than calibrated ones but also actively misleading. A Random Forest interval of [-1, +1] minutes around a 10 minute prediction looks confident, but actual delays in that scenario span tens of minutes. A user or operator relying on the interval's apparent narrowness would make decisions on bad uncertainty information. Ridge's crossover problem is similarly problematic. An interval where the point estimate lies outside its own bounds more than a fifth of the time cannot meaningfully be interpreted as a 90% confidence interval.

For prediction intervals to function as a form of local interpretability, with per-prediction interval width as a confidence signal, the interval must be calibrated correctly, which is what the results from Section 4.2.2 show. Local interpretability via interval width is real only when the interval mechanism captures genuine uncertainty. LightGBM delivers this. Random Forest's intervals communicate the wrong type of uncertainty. Ridge's intervals fail the basic monotonicity property required for the concept to apply at all. Thus, reading interval width as a confidence signal is only safe when the construction method has been verified to produce calibrated intervals.

LightGBM is the only model of the three that produces intervals fit for direct use in passenger-facing or operator-facing communications. Random Forest's intervals as constructed here cannot be used as confidence bands without further calibration and post-processing, but the trees-as-percentiles approach is a method-specific failure, not a fundamental property of Random Forest. Ridges intervals could be repaired by enforcing monotonicity constraints during fitting, but the underlying problem (linear

models struggle with collinear features) would persist.

Perhaps the most novel finding of this thesis is the three failure modes. It is not a result of model quality differences but of interval construction method differences across model families. The construction method matters at least as much as the model when asking about per-prediction confidence.

5.2 Practical Implications

The practical implications of the results of this thesis are three-fold: for passengers, operators and the field in general.

5.2.1 Passenger-Facing Communication

The 24% $RMSE$ reduction for LightGBM against the baseline (see Table 4.2) translates to meaningful improvement in long horizon ETAs, especially in the 1-3 hour range (22 minute $RMSE$ against the baseline 33 minute $RMSE$ as shown in Table 4.4). This is arguably the time frame where passengers are still making decisions on whether to leave home to catch a train, which could then be the most important horizon to improve the baseline on. Furthermore, calibrated intervals enable a user interface affordance ETAs alone cannot – a real time estimate trustworthiness indicator. Other than the glaring advantage of being able to rely on a delay span, the width of the interval also displays confidence. A narrow one indicates a strong confidence, and a wide one that a passenger should check again in a while. The caveat here is in the `On-time` delay bin, where the published ETA is already near-perfect and the model either adds no value or slightly hurts prediction. A deployed system should likely defer to the operational ETA when predicted delay is small and only correct it when the model expects meaningful deviation.

5.2.2 Operational Value

The operational value of the findings in this thesis include the results of the horizon stratification (see Section 4.3.2). The stratified skill curve identifies where the operational ETA is weakest and models strongest in comparison, the 1-3 hour band, and is also the window where operators can still act on the information including rerouting, holding connecting trains and reallocating staff, as described in Section 2.1.4.

The dominance of historic ETA features over operational metadata suggests that surfacing ETA volatility itself, how much the ETA has moved in the last few minutes, is a cheap proxy signal operators could use even without an ML model. The only

caveat here is that, for Trafikverket, it must be explicitly extracted from the current operational data using an approach similar to this thesis' (as described in Section 3.3).

5.2.3 Field Insights

As for field insights, the snapshot-keyed schema is a transferable design pattern. Any forecasting system that publishes a time varying estimate against a fixed ground truth (flight ETAs, weather forecasts, delivery estimations) could be evaluated this way, and the skill versus baseline framing generalises this thesis' objective directly.

Additionally, there is a lesson in methodology in the interval results. Quantile regression objectives (as in LightGBM) outperform constructions such as tree-percentiles and independent fits in this setting (see Table 4.3). Future work using confidence intervals should not assume that the construction is incidental or negligible.

5.3 Limitations

This section covers limitations in the framing, execution and results of this thesis, from which future research could build upon.

5.3.1 Proof-of-Concept Scope

This thesis is a proof-of-concept, because of its limited data collection period. Due to time constraints, there was only one month available for collecting data (2026-02-02 to 2026-03-02). This most likely had a large impact on seasonal variation, where the climate in Sweden at this time stayed roughly the same, thus feeding the dataset winter-only data. Performance numbers are therefore conditional on February-March 2026 conditions and should not be projected to other seasons. Furthermore, this most likely impacted all weather feature importances since they did not change much across snapshots in the training set. Similarly, the feature capturing the current month had only one possible value in the training set – 2 (February), thus rendering it utterly unimportant as a feature. Additionally, the dataset consists of only SJ long-distance trains. The models have therefore not been tested on commuter trains, freight trains or regional operators. Running models on these train types would most likely yield worse results.

With this limitation in mind, this is a feasibility study, answering what could be achieved with a collection period long enough incorporating all train types, and not a deployable production model.

5.3.2 Leakage Discovery

As explained in Section 5.1.2, the original `prev_dep_delay` feature was derived from the ground truth realised departure time, and was broadcast identically to every snapshot of the downstream event including snapshots fetched before the upstream departure actually occurred. The leaky version gave an R^2 of 0.94 and a skill of +0.64. The corrected version gives an R^2 of 0.74 and a skill of +0.24. The methodological lesson here is that in time-series settings with multi-step dependencies, leakage can hide inside features that look perfectly reasonable. Standard temporal train/test splits do not catch this as the leakage was within each row's feature vector, not between training and validation data groups. Since the leakage eventually was discovered, this is more of a contribution than a limitation.

5.3.3 Long-Horizon Skill as an Upper Bound

The 3-6 hour horizon bin is 65% of validation rows, but consists almost entirely of single row events that survived the feature engineering deduplication. This means the long horizon performance partly reflects event selection bias, where events that already had stable ETAs by the 3-6 hour mark are over-represented. Reported long horizon skill should thus be read as an optimistic upper bound, not as the expected performance on a representative random sample of all 3-6 hour forecasts.

5.3.4 Class Imbalance of Severe Delays

2.6% of validation rows are severe delays. Even with a skill of +0.29 in this bin for LightGBM, absolute errors are large (53.5 minute $RMSE$) and bias is +33.6 minutes. The model systematically under-predicts severity. Thus for the operationally most important case, even though skill is at its highest, the model is the least precise. A production system would need a tail focused training strategy to reduce this problem.

5.3.5 Snapshot Density

There is a median of 1.6 snapshots per event in the dataset. The snapshot-keyed schema was designed to enable multiple snapshot evaluations per event, but feature filtering produces a validation distribution dominated by single row events. Therefore, within-event prediction-trajectory analysis (watching a forecast sharpen as the event approaches) is not statistically supported by the current dataset. While the deduplication made to the dataset was methodologically sound in terms of space handling and time complexity, future work may benefit from revising the strategy used.

5.3.6 Categorical Encoding Asymmetry

LightGBM handles categorical features natively, but Random Forest and Ridge both require one-hot encoding of categoricals (as first mentioned in Section 3.4. This is a structural contributor to LightGBM's performance advantage, not purely a function of model capacity. A fair comparison would require target and frequency encoding for the other models, which future work could expand upon.

5.3.7 Ensemble Hyperparameter Space

Several chosen hyperparameter values for LightGBM and Random Forest (see Section 3.5.2) landed near the lower boundaries of their search ranges (for example number of leaves, maximum depth and feature subsampling), suggesting the models favoured aggressive regularisation. This is consistent with a dataset where a small set of features (operational ETA and ETA history) carries the bulk of the predictive signal. Additional model capacity provides diminishing returns and risks overfitting. Ridge's selected regularisation strength (100) similarly reflects aggressive shrinkage, consistent with the collinearity between operational ETA and ETA-history features discussed in Section 2.3.5. Since this is the case for some of the features in the ensemble methods, a different, lower value hyperparameter space may have proven more robust.

5.4 Future Work

This thesis is a feasibility study, so future work falls into two camps – closing gaps that the proof-of-concept scope left open, and extending the contributions into directions not pursued here. The subsections below are grouped thematically rather than following the limitations exactly (which can be found in Section 5.3).

5.4.1 Extending the Dataset

This section builds on the limitations mentioned in Sections 5.3.1, 5.3.4 and 5.3.5.

Future work could likely benefit from a collection period spanning over multiple seasons, at least a year, to capture seasonal variation (as mentioned in Section 5.3.1). Similarly, other operator types such as commuter trains, regional operators and freight trains have different delay dynamics, potentially prompting further investigation.

The deduplication schema discussed in Section 5.3.5 could be worth revisiting, as preserving more intermediate snapshots, even unchanged ones, would enable proper within-event trajectory analysis.

Targeted oversampling or stratified sampling of severe-delay events to address the class imbalance flagged in Section 5.3.4 also warrants further investigation.

5.4.2 Modelling Extensions

This section builds on discussion and limiations mentioned in Sections 5.1.1, 5.3.6 and 5.3.7.

Future work could benefit from fair categorical encoding comparison, as per the limitation discussed in Section 5.3.6. Examples of possible solutions to the limitation are target encoding or frequency encoding for Ridge and Random Forest, in order to isolate model capacity from encoding advantage.

Moreover, tail-focused training strategies could be looked into, such as specialised tail models for the `Severe delay` bin, which would possible provide a solution for the limitations mentioned in Section 5.1.1.

Since the snapshot schema is inherently temporal, recurrent or transformer-based models could exploit the snapshot sequence directly rather than the engineered ETA history features used here. Hierarchical or multi-task models could also prove beneficial here, predicting deviation together with related quantities such as delay at next station or cancellation probability, so the model shares structure across related signals.

Finally, a deployable system would need to update continually as new snapshots arrive. Comparing batch retraining cadences to true online learners is an open question prompting investigation.

5.4.3 Uncertainty and Prediction Intervals

This section builds on the discussion from Sections 5.1.3 and 5.2.3.

Future work could benefit from exploring repair strategies for the two failed prediction interval methods (as discussed in Section 5.1.3).

Worth exploration is also cross-method comparisons of intervals on equal footing. Applying conformal prediction as a wrapper to all three models would isolate the model family contribution from the interval construction contribution.

Coverage of the intervals was reported overall (see Table 4.3), but stratifying by horizon and by delay severity would reveal whether LightGBM's 90.7% coverage holds in the severe delay tail, which is where local interpretability might matter most.

Instead of two quantiles used as the upper and lower bounds (see Section 4.2.2), future work could predict full conditional distributions which subsume intervals and enable

richer UI affordances for both operators and end users.

5.4.4 Deployment and Communication

This section builds on discussion from Sections 5.2.1 and 5.2.2.

Future work can benefit from deferring to baseline logic when apt. The suggestions from Section 5.2.1 could be operationalised as a learned gating policy, asking when the operational ETA wins and when the correction should be surfaced. This itself is a small classification problem worth its own evaluation.

ETA volatility as a lightweight signal is worth exploring (as discussed in Section 5.2.2). The observation that ETA history features dominates suggests testing whether a simple, model-free volatility indicator captures most of the operator facing value.

5.4.5 Generalisation Beyond Rail

This section builds on discussion from Sections 5.2.3 and 5.3.2.

As discussed in Section 5.2.3, in domains outside of railways, the snapshot-keyed schema and the skill versus baseline framing could be applied that publish revising ETAs against fixed ground truths, including flight arrivals, package deliveries, ride sharing and weather forecasting. The leakage pattern documented in Section 5.3.2 is also a transferable methodological warning for any setting where features can be silently broadcast across snapshot time.

Chapter 6

Conclusion

This thesis investigated how ML can correct and improve on real-time ETA forecasts from Trafikverket's open API, rather than predict delays from scratch, and whether the resulting predictions can be used as calibrated uncertainty. The work is a proof-of-concept on a single month of SJ long-distance train data (2026-02-02 to 2026-03-02). This chapter restates the answers to the three research questions asked in Section 1.1 and reflects on what the contributions amount to.

6.1 Answers to the Research Questions

How accurately can train delays be predicted using supervised ML models trained on real time operational data, and in what ways do the models improve the operational estimation?

All three models beat the operational ETA on $RMSE$. Skill for Ridge, Random Forest and LightGBM are +0.10, +0.21 and +0.24 respectively. Skill is concentrated in the delay tail and at longer horizons, where in the `On-time` bin, models add no value and the baseline should dominate a deployed system. Skill also follows model complexity, with the boosting model winning by a meaningful margin.

Which features and modelling approaches contribute most to predictive performance and robustness?

The operational ETA itself, ETA history features and current horizon dominate predictive performance across all three models, validating the snapshot-keyed dataset design, whereas weather features were essentially uninformative under the collection window. The categorical-encoding asymmetry between LightGBM and Ridge/Random Forest is a structural contributor to LightGBM's lead, not just a function of model capacity.

The original `prev_dep_delay` feature illustrated how leakage can hide inside multi-step temporal features that pass standard train/test disciplines, a methodological finding in its own right.

How do prediction intervals produced by ML models differ in calibration and width, and what does this imply about their suitability for representing train delay forecast uncertainty?

The three model families produces three qualitatively distinct failure modes, the arguably most novel finding of the thesis. LightGBM's quantile regression achieved near nominal coverage (90.7%) and is the only method of the three suitable for direct passenger or operator facing communication. Random Forest's tree-prediction-percentile method severely under-covered (27.6%), and Ridges independent quantile fits produced a high bound-crossing rate of 21.85%. The construction method therefore matters at least as much as the model when constructing per-prediction confidence.

6.2 Reflection on Contributions

The stated contributions, as introduced in Section 1.2, were four bullets. First, the thesis contributes with a temporally structured dataset linking ETA snapshots to realised outcomes, realised as the snapshot-keyed schema with deduplication, transferable to any forecasting domain with revising estimates. The second contribution is a supervised learning framework for delay estimation, realised as the correction framing (operational ETA as input, delay as target), distinct from competing with the baseline from scratch. Third, the work contributes with a comparative evaluation of regression models for real time delay prediction, realised through Ridge, Random Forest and LightGBM across aggregate metrics, horizon stratification and delay-severity stratification. As the fourth contribution, the thesis provides a comparative analysis of prediction interval construction methods, realised as the three failure modes finding, with empirical evidence that quantile regression is the only well-calibrated option among the three.

6.3 Closing Remarks

Results of this thesis are conditional on a single winter month, a single operator and the other limitations documented in Section 5.3. The thesis establishes a reproducible template for further work, with priority directions including multi-season collection, fair categorical encoding, and conformal calibration of the under-performing interval methods.

This thesis demonstrates that ML can improve point predictions and communicate calibrated uncertainty using the operational systems in place. It also shows that the modelling, stratification and interval construction choices matter for practical usage of ML in train delay forecasting.

Bibliography

[1] Ivan Belcic and Cole Stryker. *What is Learning Rate in Machine Learning?* en. 2024. URL: https://www.ibm.com/think/topics/learning-rate (visited on 04/26/2026).

[2] Candice Bentéjac, Anna Csörgő, and Gonzalo Martínez-Muñoz. "A comparative analysis of gradient boosting algorithms". In: *Artificial Intelligence Review* 54 (2020), pp. 1937–1967. DOI: https://doi.org/10.1007/s10462-020-09896-5. URL: https://link.springer.com/article/10.1007/S10462-020-09896-5#citeas.

[3] James Bergstra and Yoshua Bengio. "Random Search for Hyper-Parameter Optimization". In: *Journal of Machine Learning Research* 13.10 (2012), pp. 281–305. URL: https://jmlr.org/papers/v13/bergstra12a.html.

[4] Leo Breiman. "Random Forests". In: *Machine Learning* 45.1 (2001), pp. 5–32. DOI: 10.1023/A:1010933404324. URL: https://link.springer.com/article/10.1023/A:1010933404324.

[5] Michael Chen. *How Supervised Learning Makes AI Smarter*. en. Article. 2024. URL: https://www.oracle.com/se/artificial-intelligence/machine-learning/supervised-learning/ (visited on 04/26/2026).

[6] Franz Dietrich. *Jury Theorems*. en. Encyclopedia. 2023. URL: https://plato.stanford.edu/entries/jury-theorems/#CondJuryTheo (visited on 04/21/2026).

[7] Karin Ekman. *Nya siffror: Fler tåg kommer i tid*. sv. 2025. URL: https://www.svt.se/nyheter/nya-siffror-fler-tag-kommer-i-tid (visited on 04/26/2026).

[8] Gary W Evans and Richard E Wener. "Rail commuting duration and passenger stress". In: *Health Psychology : official journal of the Division of Health Psychology, American Psychological Association* 25(3) (2006), pp. 408–412. DOI: 10.1037/0278-6133.25.3.408.

[9] Aaron Fisher, Cynthia Rudin, and Francesca Dominici. "All Models are Wrong, but Many are Useful: Learning a Variable's Importance by Studying an Entire Class of Prediction Models Simultaneously". In: *Journal of Machine Learning*

Research 20.177 (2019), pp. 1–81. URL: https://jmlr.org/papers/v20/18-760.html.

[10] Framna. "About Framna". In: (2025). URL: https://www.framna.com/about.

[11] Bowen Gao et al. "A novel deep learning model for short-term train delay prediction". In: *Information Sciences* 645 (Oct. 2023), p. 119270. ISSN: 0020-0255. DOI: 10.1016/j.ins.2023.119270. URL: https://www.sciencedirect.com/science/article/pii/S0020025523008551 (visited on 02/11/2026).

[12] Rob M.P. Goverde. "A delay propagation algorithm for large-scale railway traffic networks". In: *Transportation Research Part C: Emerging Technologies* 18.3 (2010), pp. 269–287. DOI: https://doi.org/10.1016/j.trc.2010.01.002. URL: https://www.sciencedirect.com/science/article/pii/S0968090X10000124.

[13] Rob M.P. Goverde. "A DELAY PROPAGATION ALGORITHM FOR LARGE-SCALE SCHEDULED RAIL TRAFFIC". In: *IFAC Proceedings Volumes* 39.12 (2006), pp. 245–250. ISSN: 14746670. DOI: 10.3182/20060829-3-NL-2908.00043. URL: https://linkinghub.elsevier.com/retrieve/pii/S1474667015336284 (visited on 03/02/2026).

[14] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2nd. Springer Series in Statistics. New York: Springer, 2009. DOI: 10.1007/978-0-387-84858-7. URL: https://doi.org/10.1007/978-0-387-84858-7.

[15] Rob J. Hyndman and George Athanasopoulos. *Forecasting: Principles and Practice*. 3rd. Melbourne, Australia: OTexts, 2021. URL: https://otexts.com/fpp3/.

[16] IBM. *What Is Bagging?* en. 2021. URL: https://www.ibm.com/think/topics/bagging (visited on 04/26/2026).

[17] Jonas Johansson and Henrik Hassel. "An approach for modelling interdependent infrastructures in the context of vulnerability analysis". In: *Reliability Engineering & System Safety* 95.12 (2010), pp. 1335–1344. ISSN: 0951-8320. DOI: 10.1016/j.ress.2010.06.010. URL: https://www.sciencedirect.com/science/article/pii/S0951832010001444 (visited on 03/02/2026).

[18] Jonas Johansson, Henrik Hassel, and Alexander Cedergren. "Vulnerability analysis of interdependent critical infrastructures: case study of the Swedish railway system". In: *International Journal of Critical Infrastructures* 7.4 (2011), pp. 289–316. ISSN: 1475-3219. DOI: 10.1504/IJCIS.2011.045065. URL: https://www.inderscienceonline.com/doi/full/10.1504/IJCIS.2011.045065 (visited on 03/02/2026).

[19] Guolin Ke et al. "LightGBM: A Highly Efficient Gradient Boosting Decision Tree". In: *Advances in Neural Information Processing Systems 30*. Ed. by I. Guyon

et al. Curran Associates, Inc., 2017, pp. 3146–3154. URL: https://proceedings.neurips.cc/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html.

[20] Roger Koenker and Gilbert Bassett. "Regression Quantiles". In: *Econometrica* 46.1 (1978), pp. 33–50. DOI: 10.2307/1913643. URL: https://www.jstor.org/stable/1913643.

[21] Elena Kosourova. *RMSE Explained: A Guide to Regression Prediction Accuracy*. en. Article. 2025. URL: https://www.datacamp.com/tutorial/rmse (visited on 04/26/2026).

[22] Zihao Li et al. "Explaining tree ensembles through single decision trees". In: *Information Fusion* 123 (2025), p. 103244. ISSN: 1566-2535. DOI: 10.1016/j.inffus.2025.103244. URL: https://www.sciencedirect.com/science/article/pii/S1566253525003173 (visited on 04/26/2026).

[23] Qianyi Liu et al. "Prediction of high-speed train delay propagation based on causal text information". In: *Railway Engineering Science* 31.1 (2023), pp. 89–106. ISSN: 2662-4753. DOI: 10.1007/s40534-022-00286-x. URL: https://doi.org/10.1007/s40534-022-00286-x (visited on 03/03/2026).

[24] Scott M. Lundberg and Su-In Lee. "A Unified Approach to Interpreting Model Predictions". In: *Advances in Neural Information Processing Systems 30*. Ed. by I. Guyon et al. Curran Associates, Inc., 2017, pp. 4765–4774. URL: https://proceedings.neurips.cc/paper_files/paper/2017/hash/8a20a8621978632d76c43dfd28b6
Abstract.html.

[25] Nicolai Meinshausen. "Quantile Regression Forests". In: *Journal of Machine Learning Research* 7.35 (2006), pp. 983–999. URL: https://jmlr.org/papers/v7/meinshausen06a.html.

[26] Tim Miller. "Explanation in Artificial Intelligence: Insights from the Social Sciences". In: *Artificial Intelligence* 267 (Feb. 2019), pp. 1–38. DOI: 10.1016/j.artint.2018.07.007. URL: https://www.sciencedirect.com/science/article/pii/S0004370218305988.

[27] Grace Mukunzi and Carl-William Palmqvist. "The impact of railway incidents on train delays: A case of the Swedish Railway Network". In: *Journal of Rail Transport Planning & Management* 30 (2024), p. 100445. ISSN: 2210-9706. DOI: 10.1016/j.jrtpm.2024.100445. URL: https://www.sciencedirect.com/science/article/pii/S2210970624000155 (visited on 02/16/2026).

[28] Jacob Murel and Eda Kavlakoglu. *What Is Ridge Regression?* en. 2023. URL: https://www.ibm.com/think/topics/ridge-regression (visited on 04/26/2026).

[29] Luca Oneto et al. "Train Delay Prediction Systems: A Big Data Analytics Perspective". In: *Big Data Research* 11 (2018), pp. 54–64. ISSN: 2214-5796. DOI:

https://doi.org/10.1016/j.bdr.2017.05.002. URL: https://www.sciencedirect.com/science/article/pii/S2214579617300060.

[30] Ahmed M Salih and Yuhe Wang. *Are Linear Regression Models White Box and Interpretable?* 2024. DOI: 10.48550/arXiv.2407.12177. arXiv: 2407.12177 [cs.LG]. URL: https://arxiv.org/abs/2407.12177.

[31] scikit-learn. *Decision Trees*. en. Documentation. 2026. URL: https://scikit-learn/stable/modules/tree.html (visited on 04/26/2026).

[32] Glenn Shafer and Vladimir Vovk. "A Tutorial on Conformal Prediction". In: *Journal of Machine Learning Research* 9.12 (2008), pp. 371–421. URL: https://jmlr.org/papers/v9/shafer08a.html.

[33] Lloyd S. Shapley. "A Value for n-Person Games". In: *Contributions to the Theory of Games, Volume II*. Ed. by H. W. Kuhn and A. W. Tucker. Annals of Mathematics Studies 28. Princeton, NJ: Princeton University Press, 1953, pp. 307–317. DOI: 10.1515/9781400881970-018. URL: https://doi.org/10.1515/9781400881970-018.

[34] Geeta Sikka, Arvinder Kaur, and Moin Uddin. "Estimating function points: Using machine learning and regression models". In: *2010 2nd International Conference on Education Technology and Computer*. Vol. 3. 2010, pp. V3-52-V3–56. DOI: 10.1109/ICETC.2010.5529600.

[35] Roger W. Sinnott. "Virtues of the Haversine". In: *Sky and Telescope* 68.2 (1984), p. 158.

[36] Jasper Snoek, Hugo Larochelle, and Ryan P. Adams. "Practical Bayesian Optimization of Machine Learning Algorithms". In: *Advances in Neural Information Processing Systems 25*. Ed. by F. Pereira et al. Curran Associates, Inc., 2012, pp. 2951–2959. URL: https://papers.nips.cc/paper/4522-practical-bayesian-optimization-of-machine-learning-algorithms.

[37] Thomas Spanninger. "Uncertainty-aware Predictions of Train Delay in Real-time". PhD thesis. ETH Zurich, 2023. DOI: 10.3929/ETHZ-B-000658746. URL: http://hdl.handle.net/20.500.11850/658746 (visited on 03/04/2026).

[38] Carolin Strobl et al. "Bias in Random Forest Variable Importance Measures: Illustrations, Sources and a Solution". In: *BMC Bioinformatics* 8.1 (2007), p. 25. DOI: 10.1186/1471-2105-8-25. URL: https://doi.org/10.1186/1471-2105-8-25.

[39] Donald Sussman and Michael Coplen. "Fatigue and alertness in the United States railroad industry part I: the nature of the problem". In: *Transportation Research Part F: Traffic Psychology and Behaviour* 3.4 (2000), pp. 211–220. DOI: https://doi.org/10.1016/S1369-8478(01)00005-5. URL: https://www.sciencedirect.com/science/article/pii/S1369847801000055.

[40] Trafikverket. “Datamodell”. In: (2025). URL: https://data.trafikverket.se/documentation/datacache/data-model.

[41] Trafikverket. “Nio av tio tåg i tid i november när punktligheten steg”. In: (2025). URL: https://www.trafikverket.se/om-oss/nyheter/nationella-nyheter/2025/december/nio-av-tio-tag-i-tid-i-november-nar-punktligheten-steg/.

[42] Vamsikd. *Basic Understanding of Loss Functions and Evaluation Metrics in AI: From ML to Generative AI*. en. 2025. URL: https://medium.com/@vamsikd219/basic-understanding-of-loss-functions-and-evaluation-metrics-in-ai-from-ml-to-generative-ai-957decc23aca (visited on 04/26/2026).

[43] Huy Minh Vo, Jyh-Bin Yang, and Veerakumar Rangasamy. “A comprehensive review of delay analysis techniques for solving concurrent delays”. In: *Engineering, Construction and Architectural Management* 32.5 (2024), pp. 3495–3521. ISSN: 0969-9988. DOI: 10.1108/ECAM-10-2023-1022. eprint: https://www.emerald.com/ecam/article-pdf/32/5/3495/10069396/ecam-10-2023-1022.pdf. URL: https://doi.org/10.1108/ECAM-10-2023-1022.

[44] Josef Waples. *Mean Absolute Error Explained: Measuring Model Accuracy*. en. Article. 2025. URL: https://www.datacamp.com/tutorial/mean-absolute-error (visited on 04/26/2026).

[45] *What is lasso regression?* en. 2024. URL: https://www.ibm.com/think/topics/lasso-regression (visited on 04/26/2026).

[46] Matthew Wiens et al. “A Tutorial and Use Case Example of the eXtreme Gradient Boosting (XGBoost) Artificial Intelligence Algorithm for Drug Development Applications”. In: *Clinical and Translational Science* 18.3 (2025). e70172 CTS-2024-0552C, e70172. DOI: https://doi.org/10.1111/cts.70172. eprint: https://ascpt.onlinelibrary.wiley.com/doi/pdf/10.1111/cts.70172. URL: https://ascpt.onlinelibrary.wiley.com/doi/abs/10.1111/cts.70172.

[47] Jared Wilber. *Linear Regression*. en. Article. Sept. 2022. URL: https://mlu-explain.github.io/linear-regression/ (visited on 04/21/2026).

[48] Jared Wilber. *Neural Networks*. en. 2023. URL: https://mlu-explain.github.io/neural-networks/ (visited on 04/26/2026).

[49] Jared Wilber. *Precision and Recall*. en. 2022. URL: https://mlu-explain.github.io/precision-recall/ (visited on 04/26/2026).

[50] Jared Wilber. *ROC and AUC*. en. Article. 2022. URL: https://mlu-explain.github.io/roc-auc/ (visited on 04/26/2026).

[51] Jared Wilber and Lucía Santamaría. *Decision Trees*. en. Article. 2026. URL: https://mlu-explain.github.io/decision-tree/ (visited on 04/21/2026).

[52] Jenny Yeon and Jared Wilber. *The Random Forest Algorithm*. en. Article. 2026. URL: https://mlu-explain.github.io/random-forest/ (visited on 04/21/2026).

Appendix A

Additional Tables and Figures

Table A.1: Meteorological parameters used from SMHI.

Parameter	**API code**	**Description**	**Unit**	**Frequency**
Temperature	1	Air temperature	°C	Hourly
Average Temperature	2	Average air temperature	°C	Daily
Precipitation	5	Precipitation	mm	Every 15 min
Precipitation Sum	7	Sum of precipitation	mm	Daily
Air Pressure	9	Air pressure at sea level	hPa	Hourly
Average Wind Speed	10	Average wind speed	m/s	Hourly
Wind Gust	21	Maximum wind speed	m/s	Hourly
Dew Point	39	Dew point temperature	°C	Hourly
Ground condition	40	Ground condition	Code	Daily

Table A.2: Python libraries used, and their area of usage.

Usage Area	**Libraries**
Data handling	`pandas` `numpy`
Database access	`sqlite3`
Modelling	`scikit-learn` (Ridge, RandomForestRegressor, StandardScaler, RandomizedSearchCV, TimeSeriesSplit) `lightgbm`
Evaluation & Interpretation	`scikit-learn` (metrics, permutation_importance) `shap`
Plotting & Figures	`matplotlib`

Collection & Scheduling	`requests`
	`python-crontab`
	`python-dotenv`

Table A.3: Permutation Importance (5000 samples) – LightGBM (full list).

Feature	$\Delta RMSE$ (mean)	Std
eta_delay_minutes	8.4319	0.0973
horizon_minutes	4.0471	0.1511
eta_delay_lag1	1.7801	0.0421
eta_delay_delta	1.6200	0.0489
location_signature	1.1001	0.0259
eta_delay_rolling_mean_3	0.3837	0.0195
eta_delay_lag2	0.3612	0.0262
temp_avg_daily	0.0856	0.0085
hour	0.0658	0.0371
is_arrival	0.0622	0.0315
prev_dep_delay	0.0504	0.0193
minutes_since_last_snapshot	0.0412	0.0152
temp_instant	0.0323	0.0063
precip_daily	0.0281	0.0064
dew_point	0.0150	0.0070
wind_gust	0.0042	0.0057
precip_15min	0.0004	0.0001
operator	0.0000	0.0000
month	0.0000	0.0000
day_of_week	-0.0147	0.0460
n_snapshots_seen	-0.0266	0.0141

Table A.4: Permutation Importance (5000 samples) – Random Forest (full list).

Feature	ΔRMSE (mean)	Std
eta_delay_minutes	7.2460	0.1061
horizon_minutes	3.0161	0.1067
eta_delay_lag1	1.9511	0.0533
eta_delay_delta	1.5953	0.0482
eta_delay_rolling_mean_3	1.0719	0.0327
eta_delay_lag2	0.9382	0.0469
minutes_since_last_snapshot	0.1602	0.0335
temp_avg_daily	0.0909	0.0099

continued on next page

Table A.4 – *continued from previous page*

Feature	ΔRMSE (mean)	Std
dew_point	0.0640	0.0108
prev_dep_delay	0.0500	0.0233
temp_instant	0.0318	0.0056
is_arrival	0.0198	0.0048
hour	0.0195	0.0245
precip_daily	0.0190	0.0058
location_signature_OTHER	0.0125	0.0091
wind_gust	0.0095	0.0054
location_signature_Arnc	0.0066	0.0006
location_signature_Gä	0.0064	0.0014
location_signature_Hm	0.0031	0.0007
location_signature_U	0.0030	0.0012
location_signature_Cst	0.0016	0.0005
precip_15min	0.0002	0.0000
location_signature_Öx	0.0002	0.0001
location_signature_N	0.0002	0.0002
location_signature_Hpbg	0.0002	0.0003
location_signature_Av	0.0001	0.0002
location_signature_Sk	0.0001	0.0001
location_signature_Gas	0.0001	0.0002
location_signature_Jö	0.0000	0.0000
location_signature_Snu	0.0000	0.0000
location_signature_Kp	0.0000	0.0000
location_signature_T	0.0000	0.0000
location_signature_Hrc	0.0000	0.0000
location_signature_Fm	0.0000	0.0000
location_signature_Thn	0.0000	0.0001
location_signature_Hka	0.0000	0.0000
location_signature_Fby	0.0000	0.0000
location_signature_Söö	0.0000	0.0000
location_signature_Bål	0.0000	0.0000
location_signature_Vå	0.0000	0.0000
location_signature_Kla	0.0000	0.0000
location_signature_Arb	0.0000	0.0000

continued on next page

Table A.4 – *continued from previous page*

Feature	ΔRMSE (mean)	Std
location_signature_Ör	0.0000	0.0000
location_signature_Mu	0.0000	0.0000
month	0.0000	0.0000
location_signature_Svg	0.0000	0.0000
location_signature_Ss	0.0000	0.0000
location_signature_Vaa	0.0000	0.0000
operator_SJ	0.0000	0.0000
location_signature_Yb	-0.0000	0.0000
location_signature_Ho	-0.0000	0.0000
location_signature_Sth	-0.0000	0.0000
location_signature_Öb	-0.0000	0.0000
location_signature_Än	-0.0000	0.0000
location_signature_Bry	-0.0000	0.0000
location_signature_Vg	-0.0000	0.0000
location_signature_Gm	-0.0000	0.0000
location_signature_Bhs	-0.0000	0.0000
location_signature_Ep	-0.0000	0.0000
location_signature_Th	-0.0000	0.0000
location_signature_Kde	-0.0000	0.0000
location_signature_Les	-0.0000	0.0000
location_signature_F	-0.0000	0.0001
location_signature_Vgå	-0.0000	0.0000
location_signature_A	-0.0000	0.0000
location_signature_K	-0.0001	0.0004
location_signature_Lp	-0.0001	0.0001
location_signature_Nr	-0.0001	0.0001
location_signature_G	-0.0008	0.0002
day_of_week	-0.0069	0.0150
n_snapshots_seen	-0.0488	0.0142

Table A.5: Permutation Importance (5000 samples) – Ridge (full list).

Feature	ΔRMSE (mean)	Std
eta_delay_lag1	6.2322	0.0929
eta_delay_minutes	5.5869	0.0864
eta_delay_rolling_mean_3	0.5717	0.0440
temp_avg_daily	0.3164	0.0199
horizon_minutes	0.2022	0.0176
eta_delay_lag2	0.1250	0.0065
prev_dep_delay	0.0954	0.0089
n_snapshots_seen	0.0893	0.0040
minutes_since_last_snapshot	0.0667	0.0117
eta_delay_delta	0.0601	0.0186
is_arrival	0.0486	0.0123
hour	0.0437	0.0103
location_signature_Gä	0.0186	0.0061
dew_point	0.0158	0.0023
location_signature_U	0.0153	0.0051
temp_instant	0.0151	0.0016
location_signature_Hm	0.0121	0.0062
location_signature_Arnc	0.0115	0.0022
location_signature_OTHER	0.0111	0.0029
location_signature_Cst	0.0109	0.0054
precip_daily	0.0079	0.0043
location_signature_Hka	0.0058	0.0019
location_signature_Hpbg	0.0045	0.0011
location_signature_Fm	0.0045	0.0035
location_signature_Fby	0.0043	0.0033
location_signature_Th	0.0040	0.0021
location_signature_Hrc	0.0039	0.0029
location_signature_Vaa	0.0038	0.0021
location_signature_Sth	0.0035	0.0018
location_signature_Vgå	0.0033	0.0009
location_signature_Jö	0.0032	0.0033
location_signature_Snu	0.0029	0.0009
location_signature_A	0.0029	0.0009
location_signature_F	0.0027	0.0024

continued on next page

Table A.5 – *continued from previous page*

Feature	ΔRMSE (mean)	Std
location_signature_Mu	0.0027	0.0033
location_signature_Thn	0.0018	0.0016
location_signature_Svg	0.0018	0.0015
location_signature_Vg	0.0016	0.0047
location_signature_Ss	0.0014	0.0026
location_signature_Bry	0.0014	0.0018
location_signature_Bhs	0.0012	0.0012
location_signature_Ho	0.0012	0.0012
location_signature_Sk	0.0011	0.0012
location_signature_Yb	0.0010	0.0019
location_signature_Bål	0.0006	0.0007
location_signature_Söö	0.0006	0.0003
location_signature_Nr	0.0006	0.0019
location_signature_Ep	0.0006	0.0016
location_signature_Les	0.0005	0.0004
location_signature_Än	0.0004	0.0006
location_signature_Kde	0.0003	0.0013
location_signature_G	0.0003	0.0009
location_signature_K	0.0003	0.0019
location_signature_Gm	0.0002	0.0018
location_signature_Gas	0.0001	0.0003
location_signature_Kla	0.0001	0.0010
location_signature_Av	0.0001	0.0028
location_signature_T	0.0000	0.0001
location_signature_Ör	0.0000	0.0004
month	0.0000	0.0000
operator_SJ	0.0000	0.0000
wind_gust	-0.0000	0.0000
location_signature_Vå	-0.0000	0.0002
location_signature_N	-0.0003	0.0005
location_signature_Arb	-0.0004	0.0005
location_signature_Öb	-0.0004	0.0015
precip_15min	-0.0005	0.0002
location_signature_Kp	-0.0007	0.0004

continued on next page

Table A.5 – *continued from previous page*

Feature	ΔRMSE (mean)	Std
`location_signature_Lp`	-0.0010	0.0016
`location_signature_Öx`	-0.0017	0.0015
`day_of_week`	-0.0057	0.0017

Appendix B

Example Gradient Boosting Process

Step-by-step, a simplified Gradient Boosting process can look as follows:

Table B.1: Example training data

Train	**Temperature**	**Station**	**Realised Delay**
A	-2	Göteborg	8 min
B	5	Stockholm	2 min
C	-1	Göteborg	7 min
D	3	Stockholm	3 min

If the training data, features and labels were as displayed above in Table B.1, the model would start by predicting the mean of all labels:

$$f_0 = \frac{8 + 2 + 7 + 3}{4} = 5 \text{ minutes}$$

So the current baseline model predicts that every train will be delayed 5 minutes. It then calculates the error of that prediction:

Table B.2: Example error calculation

Train	**Temperature**	**Station**	**Realised Delay**	**Error**
A	-2	Göteborg	8 min	8 – 5 = +3
B	5	Stockholm	2 min	2 – 5 = –3
C	-1	Göteborg	7 min	7 – 5 = +2
D	3	Stockholm	3 min	3 – 5 = –2

The model then trains a tree to learn these errors, not the original delays, meaning the

output targets of the tree will be the errors in Table B.2. In this case the tree may learn that, on average, trains in Stockholm have a prediction error of -2.5 and trains in Göteborg $+2.5$. Using this tree, the model tries to correct the baseline by applying the previously described gradient boosting formula:

Table B.3: Example formula application

Train	**Baseline**	**Tree learnings**	**Correction (with α)**	**New Prediction**
A	5	+2.5	$0.5 \times 2.5 = +1.25$	$5 + 1.25 = 6.25$
B	5	–2.5	$0.5 \times -2.5 = -1.25$	$5 - 1.25 = 3.75$
C	5	+2.5	$0.5 \times 2.5 = +1.25$	$5 + 1.25 = 6.25$
D	5	–2.5	$0.5 \times -2.5 = -1.25$	$5 - 1.25 = 3.75$

The new predictions in Table B.3 are better than predicting the average, but still far from correct. Next, the model calculates the current error again, and builds another tree for predicting those errors. This process repeats until reaching an error low enough.

Appendix C

Loss Functions Used in Classification

While MAE, RMSE and R^2 apply to regression tasks, classification models require a different set of evaluation criteria [49]. One of the more intuitive loss functions in this area is *accuracy*, which measures how many of the total predicted targets the model got right [49]. The formula can be written as

$$A = \frac{TP + TN}{TP + TN + FP + FN},$$

where the accuracy A is given by the number of true positives TP and true negatives TN, divided by all predictions (TP, TN, the number of false positives FP and false negatives FN). In this case, TP would mean the model correctly predicted a delay, TN would mean the model correctly predicted a train to be on time, FP would mean the model falsely predicted a delay, and FN would mean the model falsely predicted a train to be on time. The accuracy A is given as a number between 0 and 1, with a higher score indicating a more accurate model. A score of 1 would mean the model predicts correctly every time [49]. The critical issue with using accuracy as the only loss function is that it can be misleading when proportions of positives and negatives are skewed [49]. If 95% of trains arrive on time, and the model always guesses that they will be, it will give an accuracy of 0.95, which looks very accurate. However, it will still wrongly predict every single delay. This is called the *class imbalance problem*, and is why other metrics are needed alongside the function [49].

Another loss function that is designed to solve the class imbalance problem is *F1 Score*. It does this by focusing only on how well the model handles positives, answering how well the model balances catching positives and avoiding false positives [49]. The formula

can be written as

$$\mathrm{F1} = 2\frac{pr}{p+r},$$

where the *harmonic mean* F1 is given by the relationship of the precision p and the recall r. These variables are given by two different inner functions of positives and negatives. Precision p is written as

$$p = \frac{TP}{TP+FP},$$

which answers how many of the predicted positives were correct. Recall r is written as

$$r = \frac{TP}{TP+FN},$$

which answers how many of the positives were caught by the model. By using harmonic mean instead of a regular average, the function can punish imbalance between precision and recall – where a precision of 1 can still get a low F1 score if recall is low. In this case, a model that predicts few delays will have high precision but low recall, solving the class imbalance problem found in accuracy functions [49].

While F1 evaluates the model at one specific decision threshold – predicting positive if confidence is more than x% – the threshold is arbitrary. *AUC-ROC* (Area Under the Curve - Receiver Operating Characteristic) instead answers how well the model separates the two classes across all possible thresholds [50]. The function can be plotted as a curve on two axes, with recall as the y-axis, and the x-axis as *FPR* (False Positive Rate). The latter is written as

$$\frac{FP}{FP+TN},$$

describing how many negatives were wrongly flagged by the model. This means moving the decision threshold will trade FPR for recall, and vice versa. Tracing a curve of all thresholds, then calculating the area under that curve results in a number between 0 and 1, where 1 is a perfect separation of classes, and 0.5 is as good as a guess [50]. An example of how this plot can look is displayed in Figure C.1.

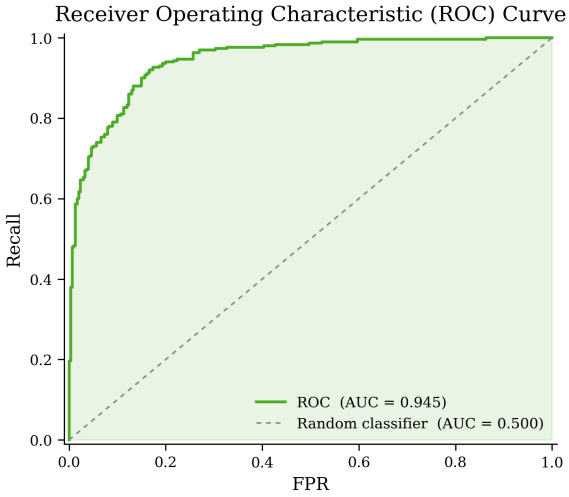


Figure C.1: Example of an AUC-ROC function plot.

While the classification model loss functions detailed above are described using binary classes, they can also be adapted to treat multi-class classification such as a low/medium/high cascading risk bin approach [49].